

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



LÊ MINH KHÔI

HỆ THỐNG QUẢN LÝ TÀI CHÍNH

ĐỒ ÁN NGÀNH
NGÀNH CÔNG NGHỆ THÔNG TIN

TP. HỒ CHÍ MINH, 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



LÊ MINH KHÔI

HỆ THỐNG QUẢN LÝ TÀI CHÍNH

Mã số sinh viên: 2351050084

Lớp: DH23IT03

ĐỒ ÁN NGÀNH

NGÀNH CÔNG NGHỆ THÔNG TIN

KHOA CÔNG NGHỆ THÔNG TIN

Giảng viên hướng dẫn: Ths. VÕ VIỆT KHOA

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Qua quá trình thực hiện đề tài đồ án ngành "Hệ thống quản lý chi tiêu và tài chính cá nhân", em cảm ơn ban giáo giám hiệu trường đại học Mở tp HCM đã tạo điều kiện thuận lợi trong việc tìm kiếm và nghiên cứu thông tin.

Em xin gửi lời cảm ơn đến giảng viên bộ môn, thầy Võ Việt Khoa, người đã tận tình hướng dẫn, chỉ bảo và góp ý cho em trong suốt quá trình thực hiện đề tài. Những kiến thức và kinh nghiệm mà thầy chia sẻ đã giúp em có thêm định hướng trong quá trình thực hiện và hoàn thiện đề tài.

Em xin chân thành cảm ơn quý thầy cô thuộc Khoa Công nghệ Thông tin đã truyền đạt cho em những kiến thức và kỹ năng cần thiết trong suốt quá trình học tập. Đây là nền tảng quan trọng để em có thể vận dụng vào việc nghiên cứu và phát triển hệ thống.

Dù đã dành nhiều thời gian nghiên cứu và thực hiện đề tài một cách nghiêm chỉnh nhưng do thời gian thực hiện cũng như kinh nghiệm thực tế còn nhiều hạn chế nên đồ án sẽ không tránh khỏi mắc phải những thiếu sót. Em rất mong nhận được những ý kiến đóng góp và nhận xét quý báu từ quý thầy cô để đề tài được hoàn thiện hơn.

Cuối cùng, em kính chúc quý thầy cô luôn mạnh khỏe, hạnh phúc và đạt được nhiều thành công trong công việc giảng dạy.

Em xin chân thành cảm ơn!

..., ngày ... tháng ... năm 2026

Sinh viên thực hiện

Khoi Le

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

TÓM TẮT ĐỒ ÁN NGÀNH

Quản lý tài chính cá nhân là một nhu cầu thiết yếu nhưng thường gặp nhiều khó khăn do thói quen ghi chép thủ công, dẫn đến thiếu sót và mất nhiều thời gian. Đồ án "Hệ thống quản lý tài chính" được thực hiện nhằm giải quyết bài toán này thông qua việc tự động hóa quy trình theo dõi dòng tiền cá nhân và gia đình. Hệ thống được phát triển dựa trên mô hình kiến trúc Client-Server, sử dụng thư viện ReactJS cho phần giao diện (Frontend) và framework Spring Boot cho phần máy chủ (Backend), kết hợp hệ quản trị cơ sở dữ liệu MySQL và được đóng gói triển khai thông qua Docker.

Điểm nổi bật của đề tài là tính năng nhập dữ liệu hàng loạt từ sao kê ngân hàng (CSV/Excel) tích hợp cơ chế tự động khử trùng lặp. Đặc biệt, hệ thống ứng dụng mô hình phân loại lai (Hybrid) kết hợp giữa tập luật từ khóa cục bộ (Rule-based) và API trí tuệ nhân tạo (Google Gemini) để tự động nhận diện và gán danh mục cho các giao dịch, giúp giảm thiểu tối đa thao tác nhập liệu thủ công. Bên cạnh đó, hệ thống cung cấp đầy đủ các chức năng cốt lõi như quản lý ví, chia sẻ tài khoản gia đình với cơ chế phân quyền chặt chẽ, thiết lập ngân sách, mục tiêu tiết kiệm, tự động gửi thông báo/cảnh báo qua Email và xuất báo cáo thống kê định dạng PDF, Excel.

Kết quả kiểm thử cho thấy hệ thống hoạt động ổn định, đảm bảo tính toàn vẹn của dữ liệu giao dịch, ngăn chặn thành công các rủi ro bảo mật phân quyền và đáp ứng trọn vẹn các yêu cầu chức năng lẫn phi chức năng đã đề ra. Đồ án đã cung cấp một công cụ quản lý tài chính chủ động, trực quan và mang tính ứng dụng thực tiễn cao.

ABSTRACT

Personal financial management is an essential need but often faces challenges due to manual tracking habits, leading to missing data and time consumption. The project "Personal Finance Management System" was implemented to address this issue by automating the workflow of tracking individual and family cash flows. The system is developed based on the Client-Server architecture, utilizing the ReactJS library for the Frontend and the Spring Boot framework for the Backend, combined with a MySQL database and deployed via Docker.

The prominent feature of this project is the bulk data import from bank statements (CSV/Excel) integrated with an automated deduplication mechanism. Notably, the system applies a Hybrid categorization model combining a local Rule-based approach and Artificial Intelligence (Google Gemini API) to automatically identify and assign categories to transactions, thereby significantly minimizing manual data entry. Additionally, the system provides core functionalities such as wallet management, family account sharing with strict role-based access control, budgeting, saving goals, automated email alerts, and the generation of statistical reports in PDF and Excel formats.

Testing results demonstrate that the system operates stably, ensuring the integrity of transaction data, successfully preventing authorization security risks, and fully meeting the defined functional and non-functional requirements. The project has delivered an active, visual, and highly practical financial management tool.

MỤC LỤC

Chương 1. GIỚI THIỆU ĐỀ TÀI	14
1.1. Bối cảnh và lý do chọn đề tài	14
1.2. Mục tiêu hệ thống	15
1.3. Đối tượng sử dụng	15
1.4. Phạm vi và chức năng chính	16
1.5. Các công nghệ liên quan	17
1.6. Bố cục báo cáo	17
Chương 2. KHẢO SÁT HIỆN TRẠNG VÀ CƠ SỞ LÝ THUYẾT	18
2.1. Khảo sát các ứng dụng quản lý chi tiêu hiện có	18
2.1.1. Money Lover	18
2.1.2. Sổ Thu Chi	18
2.1.3. Misa MoneyKeeper	19
2.1.4. Bảng so sánh các ứng dụng	19
2.1.5. Nhu cầu người dùng	20
2.2. Cơ sở lý thuyết về nhập liệu và phân tích chi tiêu	20
2.2.1. Import sao kê ngân hàng	20
2.2.2. Tự động phân loại giao dịch	21
2.2.3. Trực quan hóa dữ liệu	21
2.3. Tổng quan về các công nghệ sử dụng	21
2.3.1. Giới thiệu về SpringBoot	22
2.3.2. Giới thiệu về ReactJS	22
2.3.3. Giới thiệu về MySQL	22
2.3.4. Giới thiệu về Postman	22
2.3.5. Mô hình kiến trúc Client-Server	23
2.3.6. Giới thiệu về JWT (JSON Web Token)	23
Chương 3. PHÂN TÍCH & THIẾT KẾ HỆ THỐNG	24
3.1. Yêu cầu hệ thống	24
3.1.1. Yêu cầu chức năng	24

3.1.2. Yêu cầu phi chức năng.....	25
3.1.3. Quy tắc nghiệp vụ	27
3.1.4. Ma trận truy vết các chức năng	28
3.2. Phân tích thiết kế	30
3.2.1. Sơ đồ use case tổng quát.....	30
3.2.2. Đặc tả use case.....	30
3.2.3. Xây dựng cơ sở dữ liệu hệ thống.....	34
3.2.4. Sơ đồ lớp hệ thống.....	34
3.2.5. Sơ đồ hoạt động.....	35
3.2.6. Sơ đồ tuần tự	38
3.3 Kiến trúc hệ thống	42
3.3. Thiết kế giao diện.....	43
Chương 4. CÀI ĐẶT & TRIỂN KHAI	46
4.1. Môi trường và công nghệ.....	46
4.2. Cấu trúc mã nguồn	48
4.3. Cài đặt cơ sở dữ liệu	49
4.4 Xây dựng các chức năng chính	50
4.4.1 Nhóm chức năng CRUD cơ bản (dại diện: Quản lý ví).....	50
4.4.2. Ghi nhận giao dịch và tính nhất quán dữ liệu	52
4.4.3. Ngân sách chi và cảnh báo tự động	54
4.4.4. Nhập sao kê giao dịch và chuẩn hóa dữ liệu (Khử trùng lặp)	56
4.4.5. Chia sẻ ví gia đình và phân quyền truy cập	58
4.5. Phần lỗi kỹ thuật đặc thù	60
4.6. Đóng gói và chuyển khai.....	63
Chương 5. KIỂM THỬ VÀ ĐÁNH GIÁ.....	64
5.1. Chiến lược kiểm thử	64
5.2. Thiết kế testcase	65
5.3. Kiểm thử tự động và CI.....	69
5.4. Ma trận truy vết yêu cầu với testcase.....	70
5.5. Kết quả và defeat	72

5.6. Đánh giá đối chiếu mục tiêu	73
5.7. Phần chưa kiểm thử được	74
Chương 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	74
6.1. Kết quả đạt được.....	74
6.2. Hạn chế còn tồn tại	75
6.3. Hướng phát triển.....	76
6.4. Bài học rút ra.....	77
TÀI LIỆU THAM KHẢO	79

DANH MỤC HÌNH

Hình 1. Sơ đồ usecase của hệ thống.....	30
Hình 2. Bảng erd của hệ thống.....	34
Hình 3. Sơ đồ lớp của hệ thống.....	35
Hình 4. Sơ đồ hoạt động uc đăng nhập	35
Hình 5 Sơ đồ hoạt động uc quản lý các phiên giao dịch.....	36
Hình 6. Sơ đồ hoạt động uc nhập dữ liệu CSV, hình ảnh	36
Hình 7. Sơ đồ hoạt động uc thiết lập ngân sách.....	37
Hình 8. Sơ đồ hoạt động uc theo dõi các chỉ tiêu.....	38
Hình 9. Sơ đồ hoạt động uc chia sẻ tài khoản	38
Hình 10. Sơ đồ tuần tự uc đăng nhập.....	39
Hình 11. Sơ đồ tuần tự uc quản lý các phiên giao dịch.....	39
Hình 12. Sơ đồ tuần tự uc nhập dữ liệu CSV, hình ảnh	40
Hình 13. Sơ đồ tuần tự uc thiết lập ngân sách.....	40
Hình 14. Sơ đồ tuần tự uc theo dõi các chỉ tiêu	41
Hình 15. Sơ đồ tuần tự uc chia sẻ tài khoản.....	41
Hình 16: Sơ đồ kiến trúc hệ thống	42
Hình 17. Wireframe trang tổng quan về tài chính.....	43
Hình 18. Wireframe sổ giao dịch.....	44
Hình 19. Wireframe sổ tiết kiệm.....	44
Hình 20. Wireframe trang ngân sách	45
Hình 21. Wireframe trang tài khoản ví	45
Hình 22: Đoạn mã then chốt nhóm 1	51
Hình 23: Bảng chứng chạy nhóm 1.....	52
Hình 24: Đoạn mã then chốt nhóm 2	53
Hình 25: Bảng chứng chạy nhóm 2.....	54
Hình 26: Đoạn mã then chốt nhóm 3	55
Hình 27: Bảng chứng chạy nhóm 3, đến % quy định.....	55
Hình 28: Bảng chứng chạy nhóm 3, vượt mức	56

Hình 29: Đoạn mã then chốt 4	57
Hình 30: Bảng chứng chạy nhóm 4, định dạng cột.....	57
Hình 31: Bảng chứng chạy nhóm 4, khử trùng lặp	58
Hình 32: Đoạn mã then chốt nhóm 5	59
Hình 33: Bảng chứng chạy nhóm 5, minh họa gia đình.....	59
Hình 34: Bảng chứng chạy nhóm 5, mời khác thành viên	60
Hình 35: Bảng chứng chạy nhóm 5, mời thành viên gia đình.....	60
Hình 36: Bảng chứng chạy phần lỗi kỹ thuật, các giao dịch đã import.....	62
Hình 37: Bảng chứng chạy phần lỗi kỹ thuật, ảnh excel import	63

DANH MỤC BẢNG

Bảng 1. Bảng so sánh các ứng dụng khảo sát	20
Bảng 2. Bảng yêu cầu về chức năng hệ thống	25
Bảng 3. Yêu cầu phi chức năng cho hệ thống	27
Bảng 4. Bảng quy tắc nghiệp vụ	28
Bảng 5. Bảng ma trận truy vết	30
Bảng 6. Đặc tả uc đăng nhập.....	31
Bảng 7. Đặc tả uc quản lý các phiên giao dịch và ví.....	32
Bảng 8. Đặc tả uc nhập dữ liệu CSV, hình ảnh.....	32
Bảng 9. Đặc tả uc thiết lập ngân sách	33
Bảng 10. Đặc tả uc theo dõi chi tiêu	33
Bảng 11. Đặc tả uc chia sẻ tài khoản	34
Bảng 12. Bảng các tầng kiến trúc của hệ thống	43
Bảng 13: Các công nghệ và phiên bản của backend	47
Bảng 14: Các công nghệ và phiên bản của frontend.....	48
<i>Bảng 15: Cấu trúc thư mục của mã nguồn</i>	<i>49</i>
Bảng 16: Bảng đối chiếu giữa thực thể trong ERD và bảng dữ liệu thật	50
Bảng 17: Bảng về biến môi trường	64
Bảng 18: Bảng testcase nhóm 1	66
Bảng 19: Bảng testcase nhóm 2	66
Bảng 20: Bảng testcase nhóm 3	67
Bảng 21: Bảng testcase nhóm 4	68
Bảng 22: Bảng testcase nhóm 5	69
Bảng 23: Bảng kiểm thử thành công unit test và CI & CD	70
Bảng 24: Ma trận truy vết testcase.....	72
Bảng 25: Bảng kết quả.....	72
Bảng 26: Bảng defeat.....	73
Bảng 27: Bảng đánh giá đối chiếu mục tiêu	74

DANH SÁCH CÁC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh đầy đủ	Ý nghĩa
API	Application Programming Interface	Giao diện lập trình ứng dụng
AI	Artificial Intelligence	Trí tuệ nhân tạo
BR	Business Rule	Quy tắc nghiệp vụ
CI	Continuous Integration	Tích hợp liên tục
CSDL		Cơ sở dữ liệu
CSV	Comma-Separated Values	Tập dữ liệu phân tách bằng dấu phẩy
DEMO	Demonstration	Môi trường dùng để trình diễn sản phẩm
DEV	Development	Môi trường phát triển dành cho lập trình viên
DOM	Document Object Model	Mô hình đối tượng tài liệu
DTO	Data Transfer Object	Đối tượng truyền dữ liệu
E2E	End-to-End	Kiểm thử toàn trình (từ đầu đến cuối)
ERD	Entity Relationship Diagram	Sơ đồ thực thể - liên kết
FR	Functional Requirement	Yêu cầu chức năng
JPA	Java Persistence API	Giao diện lập trình lưu trữ dữ liệu Java
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu dạng văn bản
JWT	JSON Web Token	Token xác thực dạng chuỗi JSON
MODULE		Khối chức năng độc lập của hệ thống
NFR	Non-Functional Requirement	Yêu cầu phi chức năng
OAuth2	Open Authorization 2.0	Giao thức ủy quyền mở phiên bản 2
ORM	Object-Relational Mapping	Kỹ thuật ánh xạ đối tượng - quan hệ
PDF	Portable Document Format	Định dạng tài liệu di động
REST/RESTful	Representational State Transfer	Kiến trúc thiết kế phần mềm cho API
SMTP	Simple Mail Transfer Protocol	Giao thức truyền tải thư tín đơn giản
SPA	Single-Page Application	Ứng dụng web trang đơn
TC	Test Case	Kịch bản kiểm thử / Ca kiểm thử
UC	Use Case	Trường hợp sử dụng

Chương 1. GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh và lý do chọn đề tài

Trong cuộc sống hằng ngày, việc quản lý tài chính của một cá nhân là một nhu cầu cần thiết nhưng lại là điều mà rất nhiều người gặp khó khăn. Phần lớn mọi người không thể trả lời chính xác câu hỏi tiền của mình đã đi đâu trong tháng vừa qua, dù thu nhập và các khoản chi tiêu vẫn diễn ra đều đặn. Nguyên nhân đến từ việc thiếu một công cụ theo dõi có hệ thống. Một số nghiên cứu gần đây cho thấy rằng khả năng lập ngân sách và tự kiểm soát mức chi tiêu ảnh hưởng trực tiếp đến sự hài lòng về tài chính của cá nhân[1] và với người dùng thường chỉ ghi nhớ đại khái hoặc ghi chép rời rạc trên giấy, sổ tay hay các ứng dụng ghi chú thông thường, dẫn đến việc bỏ sót nhiều giao dịch nhỏ lẻ nhưng nếu cộng lại thì chiếm tỷ trọng đáng kể trong tổng chi tiêu.

Bên cạnh việc ghi chép thủ công, một nguồn dữ liệu quan trọng khác là “sao kê ngân hàng”, đây là nơi ghi lại gần như đầy đủ các giao dịch chuyển khoản, thanh toán qua thẻ. Tuy nhiên, dữ liệu này thường ở dạng rời rạc theo từng tháng, từng ngân hàng khác nhau, với định dạng không đồng nhất, khiến người dùng khó có thể tự tổng hợp và phân tích nếu không có công cụ hỗ trợ. Việc đối chiếu thủ công giữa sao kê và các ghi chép cá nhân vừa mất thời gian vừa dễ sai sót.

Trên thị trường hiện nay đã có một số ứng dụng hỗ trợ quản lý chi tiêu, tiêu biểu như Money Lover hay Sổ Thu Chi, Misa MoneyKeeper [7][8][9]. Các ứng dụng này giúp người dùng ghi lại giao dịch một cách thuận tiện, có giao diện thân thiện và hỗ trợ trên di động. Tuy vậy, qua khảo sát thực tế cho thấy khả năng phân tích chuyên sâu và tự động hóa của các ứng dụng này vẫn còn hạn chế, đặc biệt là ở khâu nhập đề sao kê ngân hàng và tự động phân ra các loại giao dịch. Người dùng vẫn phải nhập liệu thủ công cho phần lớn giao dịch, điều này làm giảm đáng kể trải nghiệm và tính chính xác của dữ liệu theo thời gian.

Từ những bất cập trên, đề tài hướng đến việc xây dựng một hệ thống quản lý chi tiêu cá nhân cho phép người dùng nhập trực tiếp sao kê ngân hàng dưới dạng CSV, hệ

thống sẽ tự động phân loại các giao dịch theo danh mục phù hợp, đồng thời cung cấp các công cụ phân tích thói quen chi tiêu. Từ đó, người dùng có đầy đủ dữ liệu và góc nhìn trực quan để đưa ra các quyết định tài chính hợp lý hơn, chủ động hơn trong việc kiểm soát dòng tiền của bản thân và gia đình.

1.2. Mục tiêu hệ thống

Mục tiêu đầu tiên của hệ thống là giúp người dùng quản lý toàn diện các tài khoản tài chính cá nhân của mình, bao gồm tài khoản ngân hàng, ví tiền mặt, ví điện tử, cùng với việc ghi nhận đầy đủ các giao dịch thu và chi gắn với từng danh mục cụ thể. Người dùng có thể tự tạo, chỉnh sửa các danh mục cho phù hợp với thói quen chi tiêu riêng, thay vì bị bó buộc trong một danh sách cố định do hệ thống định sẵn.

Mục tiêu thứ hai, là khả năng phân tích thói quen chi tiêu một cách trực quan và hữu ích. Hệ thống cho phép thiết lập ngân sách theo từng danh mục theo tháng, tự động cảnh báo khi có nguy cơ vượt ngân sách, cung cấp các biểu đồ thể hiện xu hướng thu chi theo thời gian, đồng thời hỗ trợ người dùng đặt ra và theo dõi tiến độ của các mục tiêu tiết kiệm cá nhân.

Mục tiêu thứ ba, cũng là mục tiêu thiết yếu nhất là cho phép nhập sao kê ngân hàng dưới định dạng tệp tin, sau đó hệ thống sẽ tự động đọc, phân tích và phân loại từng giao dịch vào danh mục thu chi tương ứng dựa trên nội dung giao dịch, số tiền và các đặc điểm khác. Điều này giúp giảm đáng kể thời gian nhập liệu thủ công, đồng thời hạn chế tối đa việc bỏ sót giao dịch.

1.3. Đối tượng sử dụng

Người dùng chính của hệ thống là người dùng cá nhân có nhu cầu quản lý tài chính của riêng mình. Nhóm người dùng này có thể tự tạo và quản lý các tài khoản, ghi nhận hoặc nhập giao dịch, thiết lập ngân sách theo danh mục cũng như đặt ra các mục tiêu tiết kiệm cho bản thân.

Bên cạnh đó, hệ thống còn hướng đến nhóm người dùng gia đình, tức các thành viên trong cùng một hộ gia đình có nhu cầu theo dõi chi tiêu chung. Thông qua tài khoản

chia sẻ, các thành viên có thể cùng ghi nhận và theo dõi các khoản thu chi chung, giúp việc quản lý tài chính gia đình trở nên minh bạch và thuận tiện hơn.

Cuối cùng, hệ thống có vai trò quản trị viên chịu trách nhiệm quản lý người dùng, xây dựng và duy trì các danh mục mẫu để người dùng mới có thể sử dụng ngay, cũng như cấu hình các thông số vận hành chung của hệ thống.

1.4. Phạm vi và chức năng chính

Về tài khoản và giao dịch, hệ thống cho phép người dùng quản lý nhiều ví hoặc tài khoản khác nhau, ghi nhận các giao dịch thu chi, đồng thời hỗ trợ chỉnh sửa và tra cứu lại lịch sử giao dịch đã thực hiện một cách nhanh chóng.

Về danh mục thu chi, người dùng có thể tự tạo và tùy biến các danh mục theo nhu cầu cá nhân, xây dựng danh mục con để phân loại chi tiết hơn, đồng thời gán màu sắc và biểu tượng riêng cho từng danh mục nhằm tăng tính trực quan khi theo dõi.

Về ngân sách và mục tiêu, hệ thống hỗ trợ đặt ngân sách cho từng danh mục theo từng tháng, thiết lập các mục tiêu tiết kiệm cụ thể, đồng thời theo dõi tiến độ thực hiện của từng ngân sách và mục tiêu theo thời gian thực.

Về nhập liệu sao kê, hệ thống cho phép nhập dữ liệu từ tệp tin do ngân hàng cung cấp, tự động phân loại giao dịch dựa trên nội dung và đặc điểm giao dịch, đồng thời có cơ chế khử trùng lặp nhằm tránh việc một giao dịch bị ghi nhận nhiều lần khi nhập nhiều lần hoặc từ nhiều nguồn khác nhau.

Về báo cáo và biểu đồ, hệ thống cung cấp các biểu đồ thể hiện xu hướng thu chi theo từng thời gian, cơ cấu chi tiêu theo từng danh mục, cũng như khả năng so sánh chi tiêu giữa các kỳ khác nhau để người dùng nhận diện sự thay đổi trong cách chi tiêu của mình.

Về nhắc nhở, hệ thống chủ động cảnh báo khi người dùng có nguy cơ vượt ngân sách đã đặt ra, nhắc nhở các hóa đơn định kỳ sắp đến hạn thanh toán, cũng như thông báo khi các mục tiêu tiết kiệm sắp đến thời hạn hoàn thành.

Về phần hệ thống, ứng dụng đảm bảo chức năng đăng nhập và phân quyền người dùng rõ ràng theo từng vai trò, cùng với khả năng sao lưu dữ liệu cá nhân nhằm bảo vệ thông tin tài chính của người dùng trước các rủi ro mất dữ liệu.

1.5. Các công nghệ liên quan

Để hiện thực hóa các chức năng nêu trên, hệ thống sẽ xây dựng dựa trên mô hình kiến trúc Client-Server, trong đó phần phía sau (backend) sử dụng SpringBoot[2] để xử lý logic nghiệp vụ và cung cấp API, phần giao diện người dùng (frontend) được thực thi với ReactJS[3] nhằm mang lại sự trải nghiệm về tương tác một cách mượt mà và dữ liệu được lưu trữ trong hệ quản trị cơ sở dữ liệu MySQL[4], còn công cụ Postman[5] được sử dụng trong quá trình xây dựng hệ thống nhằm kiểm thử các API. Phân quyền giữa các user hệ thống hiện thực bằng JWT để triển khai.

1.6. Bố cục báo cáo

Chương 1 giới thiệu khái quát về đề tài, gồm bối cảnh và lý do chọn đề tài, mục tiêu hệ thống, đối tượng sử dụng, phạm vi và các chức năng chính, cũng như các công nghệ liên quan sẽ được sử dụng để xây dựng hệ thống. Chương 2 trình bày kết quả khảo sát một số ứng dụng quản lý chi tiêu đang có trên thị trường như Money Lover, Sổ Thu Chi, và Misa MoneyKeeper [7][8][9] từ đó rút ra nhu cầu thực tế của người dùng; đồng thời trình bày cơ sở lý thuyết liên quan đến việc nhập liệu sao kê ngân hàng, tự động phân loại giao dịch và biểu diễn dữ liệu dưới dạng trực quan, cùng với phần tổng quan chi tiết về các công nghệ sử dụng và kiến trúc Client-Server của hệ thống. Chương 3 tập trung vào việc phân tích và thiết kế hệ thống, bao gồm việc xác định yêu cầu chức năng và yêu cầu phi chức năng; xây dựng sơ đồ use case tổng quát và đặc tả chi tiết từng use case; thiết kế sơ đồ lớp và cơ sở dữ liệu; xây dựng sơ đồ tuần tự cho các luồng nghiệp vụ chính; và thiết kế giao diện cho cả phía người dùng lẫn phía quản trị. Chương 4 trình bày quá trình cài đặt và triển khai hệ thống, gồm việc thiết lập môi trường phát triển và công nghệ sử dụng; xây dựng lần lượt các chức năng chính như quản lý tài khoản và giao dịch, quản lý danh mục thu/chi, quản lý ngân sách và mục tiêu tiết kiệm, nhập dữ liệu sao kê ngân hàng, tự động phân loại giao dịch, báo cáo và biểu đồ phân tích thói quen chi tiêu, hệ thống nhắc nhở, cùng

chức năng đăng nhập và phân quyền; đồng thời trình bày việc tích hợp giữa frontend và backend, kiểm thử API bằng Postman, và triển khai ứng dụng. Chương 5 trình bày quá trình kiểm thử và đánh giá hệ thống, bao gồm các kịch bản kiểm thử đã xây dựng, kết quả kiểm thử thu được, đánh giá kết quả đạt được, những hạn chế còn lại và so sánh kết quả thực hiện với các yêu cầu ban đầu đã đề ra. Chương 6 tổng kết quá trình thực hiện đề tài và đưa ra một số hướng phát triển cho hệ thống trong thời gian tới.

Chương 2. KHẢO SÁT HIỆN TRẠNG VÀ CƠ SỞ LÝ THUYẾT

2.1. Khảo sát các ứng dụng quản lý chi tiêu hiện có

2.1.1. Money Lover

Money Lover[7] là một ứng dụng hỗ trợ quản lý chi tiêu cá nhân phổ biến, được sử dụng khá phổ biến tại Việt Nam. Ứng dụng có giao diện dễ sử dụng và cho phép người dùng tạo nhiều ví khác nhau, ghi nhận giao dịch thu chi và xem báo cáo dưới dạng biểu đồ. Money Lover cũng hỗ trợ đặt ngân sách theo tháng và có tính năng nhắc nhở hóa đơn định kỳ. Tuy nhiên, việc nhập dữ liệu từ sao kê ngân hàng còn hạn chế, chủ yếu tập trung vào việc liên kết trực tiếp với một số ngân hàng nhất định, chưa hỗ trợ linh hoạt việc nhập tệp CSV do người dùng tự tải về từ ngân hàng.

2.1.2. Sổ Thu Chi

Sổ Thu Chi [8] là một ứng dụng ghi chép chi tiêu quen thuộc với giao diện đơn giản và dễ sử dụng, phù hợp với những người dùng chưa quen với công nghệ. Ứng dụng cho phép ghi nhận nhanh các khoản thu chi hằng ngày, phân loại theo danh mục có sẵn và theo dõi lịch sử giao dịch theo ngày và tháng. Điểm mạnh của Sổ Thu Chi nằm ở “sự tối giản” trong thao tác nhập liệu. Tuy nhiên, đây cũng chính là điểm hạn chế lớn nhất khi ứng dụng gần như không có khả năng phân tích chuyên sâu, không hỗ trợ nhập dữ liệu từ sao kê ngân hàng, và người dùng phải tự nhập tay toàn bộ giao dịch, dẫn đến việc dễ bỏ sót hoặc nhập sai khi khối lượng giao dịch lớn.

2.1.3. Misa MoneyKeeper

MISA MoneyKeeper [9], thường được biết đến với tên gọi Sổ Thu Chi MISA là ứng dụng quản lý tài chính cá nhân được phát triển bởi Công ty Cổ phần MISA và hiện đang là một ứng dụng khá phổ biến tại Việt Nam, với lượng người dùng lên đến hàng triệu người. So với Money Lover và Sổ Thu Chi, MISA MoneyKeeper có phần vượt trội hơn về độ đa dạng tính năng: ứng dụng cho phép tạo không giới hạn tài khoản tiền, thiết lập hạn mức chi tiêu theo từng danh mục kèm cảnh báo khi vượt mức, lập kế hoạch dự thu dự chi cho các khoản định kỳ như tiền nhà hay học phí, quản lý khoản vay và cho vay, quy đổi tỷ giá ngoại tệ, đồng thời hỗ trợ ghi chép chi tiêu riêng theo từng chuyến đi hoặc sự kiện. Ứng dụng cũng cho phép xuất báo cáo dưới dạng Excel hoặc PDF và có tính năng chia sẻ sổ chi tiêu để các thành viên trong gia đình cùng theo dõi khoản chi chung. Ứng dụng cũng hỗ trợ trong việc phân chia các khoản chi khi chụp hóa đơn, giúp người dùng dễ dàng hơn trong việc phân loại. Ngoài ra, nhiều tính năng nâng cao như dự thu dự chi, quản lý khoản vay hay quy đổi ngoại tệ

Tuy có nhiều tính năng phong phú, MISA MoneyKeeper vẫn chưa giải quyết được vấn đề cốt lõi mà đề tài hướng tới, đó là khả năng import trực tiếp file sao kê ngân hàng dưới dạng CSV. Việc ghi nhận giao dịch trong ứng dụng chủ yếu vẫn dựa vào nhập tay hoặc quét hóa đơn, còn việc phân loại giao dịch phần lớn do người dùng tự chọn danh mục theo thao tác thủ công chứ chưa có cơ chế tự động nhận diện dựa trên nội dung sao kê ngân hàng.

2.1.4. Bảng so sánh các ứng dụng

Từ nội dung khảo sát ba ứng dụng nêu trên, có thể tổng hợp một số tiêu chí so sánh chính như sau:

Tiêu chí	Money Lover	Sổ Thu Chi	MISA MoneyKeeper
Ghi nhận giao dịch thủ công	Có	Có	Có
Import sao kê CSV tự do	Không hỗ trợ	Không hỗ trợ	Không hỗ trợ
Tự động phân loại giao dịch theo nội dung	Có	Không có	Có

Tiêu chí	Money Lover	Sổ Thu Chi	MISA MoneyKeeper
Đặt ngân sách theo danh mục, cảnh báo vượt mức	Có	Không có	Có
Biểu đồ, báo cáo xu hướng thu chi	Có	Cơ bản	Có, khá chi tiết
Chia sẻ tài khoản/sổ chi tiêu gia đình	Có	Không có	Có
Mục tiêu tiết kiệm	Có	Không có	Có

Bảng 1. Bảng so sánh các ứng dụng khảo sát

Bảng so sánh cho thấy điểm chung đáng chú ý giữa cả ba ứng dụng: dù mức độ đầy đủ tính năng khác nhau, không ứng dụng nào hỗ trợ người dùng import trực tiếp và tự do file sao kê ngân hàng dạng CSV, cũng như chưa có cơ chế tự động phân loại giao dịch dựa trên nội dung sao kê. Đây chính là khoảng trống chức năng rõ ràng nhất mà cả ba ứng dụng phổ biến hiện nay đều chưa đáp ứng.

2.1.5. Nhu cầu người dùng

Từ việc khảo sát ba ứng dụng tiêu biểu là Money Lover, Sổ Thu Chi và MISA MoneyKeeper, nhu cầu của người dùng không chỉ là ghi lại các khoản chi tiêu mà còn cần nhiều chức năng như ghi chép chi tiêu, mà còn hướng đến việc giảm thiểu công sức nhập liệu thủ công thông qua khả năng nhập dữ liệu từ sao kê ngân hàng, tự động phân loại giao dịch có độ chính xác tốt, kết hợp với các công cụ phân tích và cảnh báo để hỗ trợ người dùng chủ động kiểm soát tài chính cá nhân. Trong khi Money Lover và MISA MoneyKeeper đã làm khá tốt phần ngân sách, cảnh báo và biểu đồ, còn Sổ Thu Chi mạnh ở sự đơn giản, thì cả ba đều chưa giải quyết được bài toán nhập liệu tự động từ sao kê ngân hàng. Đây chính là điểm mà các ứng dụng hiện tại chưa đáp ứng tốt, đồng thời cũng là cơ sở để hệ thống trong đề tài này xác định rõ hướng phát triển các chức năng của mình.

2.2. Cơ sở lý thuyết về nhập liệu và phân tích chi tiêu

2.2.1. Import sao kê ngân hàng

Sao kê ngân hàng là bản ghi chi tiết các giao dịch phát sinh trên tài khoản của khách hàng trong một khoảng thời gian nhất định, thường được ngân hàng cung cấp dưới định dạng file Excel/CSV khi người dùng tải về từ ứng dụng ngân hàng hoặc yêu cầu tại quầy giao dịch. Mỗi tệp sao kê thường bao gồm các trường thông tin cơ bản như ngày giao dịch, nội dung hoặc diễn giải giao dịch, số tiền ghi nợ hoặc ghi có, và số dư sau giao dịch. Tuy nhiên, định dạng cụ thể của các trường này có thể khác nhau tùy theo từng ngân hàng, đòi hỏi hệ thống phải có cơ chế đọc và chuẩn hóa dữ liệu linh hoạt, có khả năng nhận diện đúng cấu trúc file để trích xuất chính xác thông tin giao dịch trước khi đưa vào cơ sở dữ liệu.

2.2.2. Tự động phân loại giao dịch

Sau khi dữ liệu giao dịch được trích xuất từ sao kê, một trong những bài toán quan trọng là tự động xác định danh mục phù hợp cho từng giao dịch, ví dụ như ăn uống, di chuyển, mua sắm hay hóa đơn sinh hoạt. Cách tiếp cận phổ biến là dựa trên việc phân tích nội dung diễn giải giao dịch, đối chiếu với một tập từ khóa hoặc quy tắc được xây dựng sẵn cho từng danh mục. Khi nội dung giao dịch chứa các từ khóa đặc trưng, hệ thống sẽ tự động gán giao dịch đó vào danh mục tương ứng. Ngoài ra, hệ thống cũng có thể ghi nhận lại các lựa chọn phân loại thủ công của người dùng trong quá khứ để dần cải thiện độ chính xác của việc phân loại tự động cho những giao dịch có nội dung tương tự về sau.

2.2.3. Trực quan hóa dữ liệu

Trực quan hóa dữ liệu là quá trình biểu diễn dữ liệu số dưới dạng hình ảnh như biểu đồ đường, biểu đồ cột hay biểu đồ tròn, giúp người dùng dễ dàng nhận diện xu hướng và cơ cấu chi tiêu của mình thay vì phải đọc qua danh sách giao dịch dạng bảng. Trong hệ thống quản lý chi tiêu, việc trực quan hóa thường được áp dụng để thể hiện xu hướng thu chi theo thời gian, cơ cấu chi tiêu theo từng danh mục trong một kỳ nhất định, cũng như mức độ hoàn thành của các ngân sách và mục tiêu tiết kiệm. Việc lựa chọn đúng loại biểu đồ cho từng loại dữ liệu đóng vai trò quan trọng trong việc giúp người dùng tiếp nhận thông tin nhanh chóng và đưa ra quyết định kịp thời.

2.3. Tổng quan về các công nghệ sử dụng

2.3.1. Giới thiệu về SpringBoot

Spring Boot là framework mã nguồn mở được xây dựng và phát triển dựa trên nền tảng Spring Framework của Java., cho phép phát triển ứng dụng backend một cách nhanh chóng nhờ cơ chế tự động cấu hình và giảm thiểu các thao tác thiết lập thủ công vốn thường gặp khi sử dụng Spring truyền thống. SpringBoot cung cấp sẵn các module hỗ trợ xây dựng RESTful API, làm việc với cơ sở dữ liệu thông qua Spring Data JPA, cũng như tích hợp cơ chế bảo mật thông qua Spring Security. Trong hệ thống quản lý chi tiêu, SpringBoot được sử dụng để xây dựng toàn bộ lớp xử lý nghiệp vụ ở phía backend, bao gồm xử lý import sao kê, phân loại giao dịch, tính toán ngân sách và cung cấp dữ liệu cho phía giao diện thông qua các API.

2.3.2. Giới thiệu về ReactJS

ReactJS là thư viện JavaScript do Meta phát triển, được sử dụng để xây dựng giao diện người dùng dựa trên các component., cho phép tái sử dụng các thành phần giao diện và quản lý trạng thái ứng dụng một cách hiệu quả. Với cơ chế Virtual DOM, ReactJS giúp việc cập nhật giao diện diễn ra nhanh chóng và mượt mà, đặc biệt phù hợp với các ứng dụng có nhiều thao tác tương tác và cập nhật dữ liệu liên tục như biểu đồ chi tiêu hay danh sách giao dịch. Trong đề tài này, ReactJS được sử dụng để xây dựng toàn bộ phần giao diện phía người dùng, từ các màn hình nhập liệu, quản lý danh mục cho đến các trang báo cáo, biểu đồ trực quan.

2.3.3. Giới thiệu về MySQL

MySQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, được sử dụng phổ biến nhờ khả năng hoạt động ổn định, hiệu suất tốt và hỗ trợ trên nhiều ngôn ngữ lập trình, cũng như framework backend, trong đó có SpringBoot. MySQL cho phép tổ chức dữ liệu theo các bảng có quan hệ với nhau, phù hợp để lưu trữ các thực thể trong hệ thống quản lý chi tiêu như người dùng, tài khoản, danh mục, giao dịch, ngân sách và mục tiêu tiết kiệm, đồng thời đảm bảo tính toàn vẹn dữ liệu thông qua các ràng buộc khóa chính và khóa ngoại giữa các bảng liên quan.

2.3.4. Giới thiệu về Postman

Postman là công cụ hỗ trợ kiểm thử API, cho phép gửi các yêu cầu HTTP đến server và kiểm tra dữ liệu trả về. Công cụ này được sử dụng trong quá trình phát triển để kiểm tra hoạt động của các API mà không cần thao tác thông qua giao diện của hệ thống. Trong quá trình xây dựng hệ thống, Postman được sử dụng để kiểm thử các API do SpringBoot cung cấp trước khi tích hợp với phần giao diện ReactJS, giúp phát hiện sớm các lỗi liên quan đến dữ liệu đầu vào, đầu ra hay logic xử lý ở phía backend, từ đó rút ngắn thời gian debug trong quá trình phát triển.

2.3.5. Mô hình kiến trúc Client-Server

Mô hình kiến trúc Client-Server - mô hình mà trong đó hệ thống chia thành hai thành phần chính: phía client đảm nhiệm hiển thị giao diện và tương tác trực tiếp cùng với người dùng, và phía server, chịu trách nhiệm xử lý logic nghiệp vụ, truy xuất và xử lý dữ liệu. Hai thành phần này giao tiếp với nhau thông qua các API theo chuẩn RESTful, sử dụng JSON làm định dạng để trao đổi dữ liệu. Trong hệ thống quản lý tài chính cá nhân, phần client được xây dựng bằng ReactJS, đảm nhận việc hiển thị thông tin và xử lý các thao tác từ người dùng, trong khi phần server được xây dựng bằng SpringBoot đảm nhiệm việc xử lý các nghiệp vụ như import sao kê, phân loại giao dịch, tính toán ngân sách, đồng thời làm việc trực tiếp với cơ sở dữ liệu MySQL. Mô hình này giúp phân chia phần giao diện và phần xử lý nghiệp vụ thành các phần riêng biệt, từ đó thuận tiện hơn trong quá trình phát triển, bảo trì và mở rộng hệ thống về sau.

2.3.6. Giới thiệu về JWT (JSON Web Token)

JWT [6] là một chuẩn mở dùng để trao đổi thông tin xác thực giữa client và server dưới dạng một chuỗi token được mã hóa, gồm ba phần: header, payload và chữ ký. Khi đăng nhập thành công, server tạo một token chứa thông tin xác định người dùng cùng vai trò của họ trong hệ thống, sau đó gửi về cho client lưu trữ. Ở các yêu cầu tiếp theo, client đính kèm token này trong header của mỗi request, giúp server thực hiện xác thực người dùng mà không cần lưu thông tin phiên đăng nhập ở phía server, thích hợp với kiến trúc RESTful vốn không lưu trạng thái (stateless). Trong hệ thống quản lý chi tiêu cá nhân, JWT sẽ được sử dụng kết hợp với Spring Security để xác

thực người dùng đăng nhập, đồng thời phân quyền truy cập giữa các vai trò người dùng cá nhân và quản trị viên..

Chương 3. PHÂN TÍCH & THIẾT KẾ HỆ THỐNG

3.1. Yêu cầu hệ thống

3.1.1. Yêu cầu chức năng

Mã	Yêu cầu chức năng
FR-01	Hệ thống cho phép người dùng có thể tạo tài khoản mới và đăng nhập bằng tài khoản đã đăng ký hoặc thông qua Google, đồng thời quản lý phiên làm việc thông qua JWT.
FR-02	Hệ thống cho phép người dùng quản lý các giao dịch và ví (tạo, chỉnh sửa, xóa) và xem danh sách các giao dịch trong ví và ghi nhận giao dịch chỉnh sửa bằng bảng lịch sử chỉnh sửa.
FR-03	Hệ thống cho phép người dùng xem thống kê tình hình thu, chi theo khoảng thời gian thông qua các biểu đồ trực quan và nhận xét phân tích từ AI.
FR-04	Hệ thống cho phép hỗ trợ sao lưu và phục hồi dữ liệu cá nhân
FR-05	Hệ thống cho phép người dùng tạo ví, tùy biến danh mục thu/chi, danh mục con, gán màu và biểu tượng
FR-06	Hệ thống cho phép người dùng nhập dữ liệu nhanh bằng import CSV/Excel; hệ thống đọc và chuẩn hóa dữ liệu từ nhiều định dạng sao kê.
FR-07	Hệ thống tự động chuẩn hóa dữ liệu giao dịch được nhập vào như ngày giao dịch, số tiền và diễn giải; cho phép người dùng ánh xạ các cột dữ liệu từ tệp sao kê về cấu trúc dữ liệu chuẩn trước khi lưu vào cơ sở dữ liệu.

Mã	Yêu cầu chức năng
FR-08	Hệ thống tự động phân loại giao dịch vào danh mục phù hợp dựa trên quy tắc/từ khóa; giao dịch không khớp quy tắc nào được đưa vào nhóm "Chưa phân loại" để người dùng gán thủ công.
FR-09	Hệ thống cho phép người dùng thiết lập ngân sách theo từng danh mục theo tháng, cấu hình ngưỡng cảnh báo, theo dõi tiến độ sử dụng ngân sách và ghi nhận ngân sách chỉnh sửa bằng bảng lịch sử chỉnh sửa
FR-10	Hệ thống cho phép người dùng đặt mục tiêu tiết kiệm (số tiền mục tiêu, thời hạn), theo dõi tiến độ thực hiện và thông báo nếu tới hạn hoặc quá hạn.
FR-11	Hệ thống tự động gửi cảnh báo khi chi tiêu có nguy cơ vượt ngân sách, nhắc nhở hóa đơn định kỳ sắp đến hạn và mục tiêu tiết kiệm sắp đến hạn, có thể ưu tiên hiển thị theo mức độ khẩn cấp.
FR-12	Hệ thống hiển thị biểu đồ xu hướng thu/chi theo thời gian, biểu đồ cơ cấu chi tiêu theo danh mục và cho phép so sánh chi tiêu giữa các kỳ khác nhau.
FR-13	Hệ thống cho phép chia sẻ một tài khoản cho nhiều thành viên trong gia đình, phân quyền truy cập (chỉnh sửa, chỉ xem) theo từng thành viên.
FR-14	Hệ thống phân quyền người dùng theo vai trò: người dùng cá nhân và quản trị viên.
FR-15	Hệ thống cho phép quản trị viên quản lý danh sách người dùng, quản lý danh mục mẫu và cấu hình các thông số vận hành chung.
FR-16	Hệ thống cho phép người dùng có thể xuất các báo cáo thành định dạng Excel hoặc PDF

Bảng 2. Bảng yêu cầu về chức năng hệ thống

3.1.2. Yêu cầu phi chức năng

Mã	Yêu cầu phi chức năng	Mức độ
NFR-01	Bảo mật: Dữ liệu tài chính cá nhân phải được bảo vệ; mật khẩu người dùng được mã hóa (BCrypt); việc xác thực và phân quyền API được thực hiện thông qua JWT kết hợp Spring Security.	Bắt buộc
NFR-02	Toàn vẹn dữ liệu: Quá trình import sao kê phải đảm bảo chính xác và chống trùng lặp; mỗi lần import được xử lý trong một giao dịch để tránh dữ liệu không nhất quán khi có lỗi xảy ra giữa chừng.	Bắt buộc
NFR-03	Hiệu năng: Các truy vấn tạo biểu đồ và báo cáo phải phản hồi trong thời gian chấp nhận được ngay cả khi dữ liệu giao dịch tích lũy trong nhiều tháng/năm.	Ưu tiên
NFR-04	Khả năng mở rộng: Kiến trúc phân tầng (Controller – Service – Repository) cho phép bổ sung chức năng mới mà không ảnh hưởng lớn đến các thành phần hiện có.	Bắt buộc
NFR-05	Khả năng sử dụng: Giao diện thân thiện, responsive, hiển thị tốt trên cả máy tính và thiết bị di động; thao tác nhập liệu và import tối giản số bước cho người dùng.	Ưu tiên
NFR-06	Độ tin cậy: Hệ thống cần được kiểm thử đầy đủ (unit test và intergation test) trước khi triển khai nhằm hạn chế lỗi ảnh hưởng đến dữ liệu tài chính của người dùng.	Bắt buộc
NFR-07	Khả năng triển khai: Hệ thống phải đóng gói và triển khai được bằng Docker, đảm bảo môi trường chạy nhất quán giữa phát triển và thực tế.	Ưu tiên

Mã	Yêu cầu phi chức năng	Mức độ
NFR-08	Khả năng bảo trì: Mã nguồn tuân thủ quy ước đặt tên và cấu trúc thư mục rõ ràng (theo README), có tài liệu API (Newman collection) hỗ trợ việc bảo trì và mở rộng về sau.	Ưu tiên

Bảng 3. Yêu cầu phi chức năng cho hệ thống

3.1.3. Quy tắc nghiệp vụ

Mã	Quy tắc nghiệp vụ
BR-01	Mỗi giao dịch phải thuộc một ví/tài khoản và được gán một danh mục thu/chi.
BR-02	Mỗi giao dịch có loại rõ ràng (thu hoặc chi) và tác động đúng số dư của tài khoản tương ứng.
BR-03	Khi import, hệ thống phải phát hiện và khừ giao dịch trùng lặp trước khi lưu.
BR-04	Ngân sách được đặt theo danh mục và theo kỳ (tháng); chi tiêu được đối chiếu với ngân sách của kỳ tương ứng.
BR-05	Ngưỡng cảnh báo vượt ngân sách (ví dụ 80%, 100%) do người dùng cấu hình.
BR-06	Chỉ chủ tài khoản (và thành viên được chia sẻ theo quyền) mới được xem và sửa giao dịch của tài khoản đó
BR-07	Danh mục đang được ít nhất một giao dịch sử dụng thì không được xóa, chỉ được phép ẩn khỏi danh sách hiển thị
BR-08	Mọi thay đổi giao dịch và ngân sách phải được ghi nhận để phục vụ đối soát.
BR-09	Giao dịch chưa khớp quy tắc phân loại được đưa vào nhóm “Chưa phân loại” để người dùng gán thủ công.
BR-10	Ưu tiên nhắc nhở theo mức khẩn: hoá đơn/mục tiêu gần đến hạn và mức vượt ngân sách lớn được xử lý trước.
BR-11	Mục tiêu tiết kiệm bắt buộc phải có số tiền mục tiêu và thời hạn hoàn thành xác định mới được tạo

Mã	Quy tắc nghiệp vụ
BR-12	Khi mời chia sẻ tài khoản, nếu email không tồn tại trong hệ thống, lời mời bị từ chối ngay, không tạo bản ghi chờ

Bảng 4. Bảng quy tắc nghiệp vụ

3.1.4. Ma trận truy vết các chức năng

Req. ID	Req. Description (Yêu cầu chức năng)	Linked BRs (Quy tắc NV)	Module	Sub-module	Test case IDs	Remarks (Ghi chú)
FR-01	Đăng ký, đăng nhập, Google OAuth, JWT	BR-12	Authentication	Login/ Register	TC-AUTH-001 đến TC-AUTH-005	Đã cấu hình Spring Security JWT.
FR-02	Quản lý giao dịch, ví & Xem/Lọc lịch sử	BR-01, BR-02, BR-06, BR-08	Transaction / Wallet	Transaction Mgt	TC-TRAN-001 đến TC-TRAN-008 TC-WALL-001 đến TC-WALL-005	Test kỹ logic bù trừ, tự động cập nhật số dư ví.
FR-03	Thống kê thu chi (Biểu đồ, AI advice)		Dashboar d	Report & Chart	TC-REPO-001 TC-REPO-002	Test tính toán tỷ lệ %.
FR-04	Sao lưu và phục hồi dữ liệu cá nhân		Settings	Backu p & Restore	TC-BACK-001 đến TC-BACK-003	Xuất/Nhập tệp JSON cấu trúc hóa thông qua BackupService.
FR-05	Tùy biến danh mục (Màu sắc, Icon), tạo ví	BR-07	Categor y / Wallet	Catego ry Mgt	TC-WALL-001 TC-CATE-001, TC-CATE-002	Test kiểm chứng logic tạo mới ví và khởi tạo danh mục.
FR-06	Nhập sao kê CSV/Excel	BR-03	Import	File Uploa d	TC-IMP-001 TC-IMP-002 TC-IMP-005	Xử lý map cột động linh hoạt, ném ngoại lệ khi sai định dạng.
FR-07	Chuẩn hóa, phát hiện	BR-03	Import	Data Valida tion	TC-IMP-001 TC-IMP-005	So khớp ngày, tiền, mô tả, ví để khử trùng.

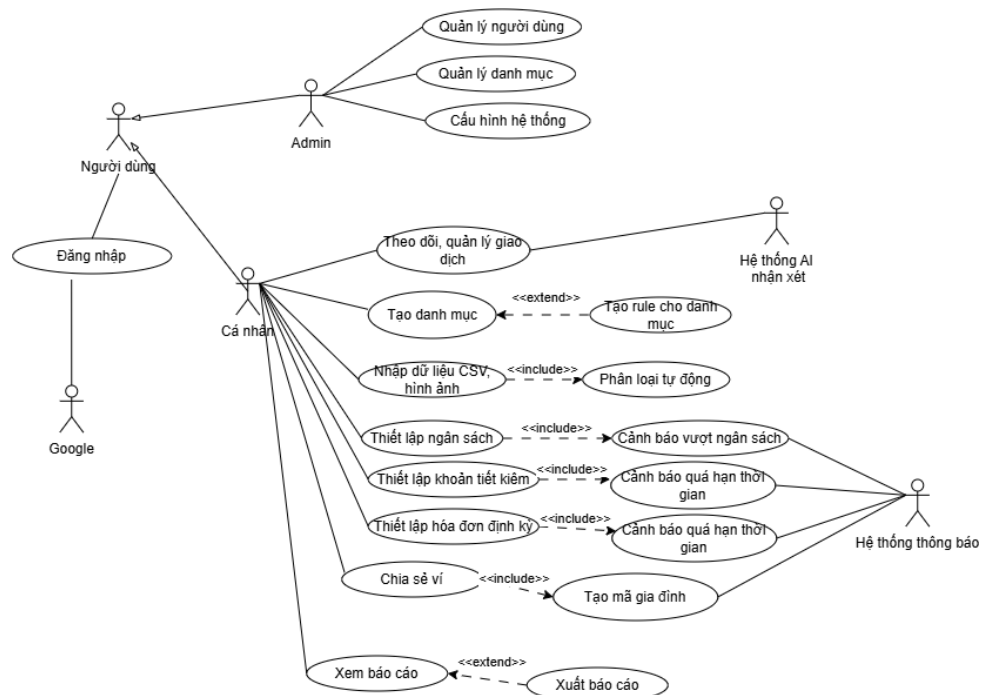
Req. ID	Req. Description (Yêu cầu chức năng)	Linked BRs (Quy tắc NV)	Module	Sub-module	Test case IDs	Remarks (Ghi chú)
	trùng lặp khi import					
FR-08	Tự động phân loại bằng Rule/AI	BR-09	Import	AI Categorization	TC-IMP-003 TC-IMP-004 TC-CATE-006, TC-CATE-007	Test ưu tiên dùng Rule cục bộ, gán "Chưa phân loại" nếu không khớp.
FR-09	Thiết lập ngân sách & ghi nhận lịch sử	BR-04, BR-05, BR-08	Budget	Budget Tracking	TC-BUDG-001 đến TC-BUDG-008	Test logic lưu lịch sử chỉnh sửa và cảnh báo.
FR-10	Đặt mục tiêu tiết kiệm, theo dõi tiến độ	BR-11	Saving Goal	Goal Tracking	TC-SAVE-001 đến TC-SAVE-007 TC-SCHED-004, TC-SCHED-005	Trừ tiền trực tiếp từ Wallet thông qua Giao dịch hệ thống.
FR-11	Cảnh báo vượt ngân sách, nhắc nhở định kỳ	BR-10	Notification / Scheduler	Alerts	TC-BUDG-006, TC-BUDG-007 TC-NOTI-006 TC-SCHED-001, TC-SCHED-002	Test luồng Cronjob chạy ngầm và tạo thông báo ưu tiên (Priority).
FR-12	Biểu đồ xu hướng, cơ cấu chi tiêu		Dashboard	Visualizations	TC-REPO-001 TC-REPO-002	Truy xuất dữ liệu tổng hợp để Frontend dùng hiển thị.
FR-13	Chia sẻ ví gia đình (View/Edit)	BR-06, BR-12	Wallet / Family	Family Sharing	TC-WALL-006 đến TC-WALL-008 TC-FAMI-001 đến TC-FAMI-011	Phân quyền kỹ lưỡng, bắt ngoại lệ chặn lời mời trái phép.
FR-14	Phân quyền (Admin, User)		Security	Authorization	TC-USER-005 TC-USER-006	Phân luồng role truy cập dựa trên Spring Security Context.

Req. ID	Req. Description (Yêu cầu chức năng)	Linked BRs (Quy tắc NV)	Module	Sub-module	Test case IDs	Remarks (Ghi chú)
FR-15	Admin Panel (Quản lý User, Cấu hình)		Admin	System Config	TC-USER-005 TC-USER-006	Quản lý trạng thái User. Cấu hình vận hành hệ thống (Chưa kiểm).
FR-16	Xuất báo cáo Excel, PDF		Report	Export	TC-REPO-003 TC-REPO-004	Sử dụng thư viện Apache POI (Excel) và OpenPDF (PDF) kết xuất mảng byte.

Bảng 5. Bảng ma trận truy vết

3.2. Phân tích thiết kế

3.2.1. Sơ đồ use case tổng quát



Hình 1. Sơ đồ usecase của hệ thống

3.2.2. Đặc tả use case

UC-01: Đăng nhập

Mục	Nội dung
Mô tả	Cho phép các actor (Cá nhân, Admin) xác thực vào hệ thống thông qua tài khoản nội bộ hoặc qua Google
Actor chính	Người dùng (Cá nhân, Admin)
Actor phụ	Google.
Tiền điều kiện	Người dùng có tài khoản trong hệ thống
Hậu điều kiện	Người dùng được cấp phiên đăng nhập (JWT) và truy cập chức năng theo vai trò
Luồng hoạt động chính	B1: Người dùng chọn đăng nhập nội bộ hoặc qua Google B2: Hệ thống yêu cầu nhập username/mật khẩu hoặc OAuth (Google) B3: Hệ thống kiểm tra thông tin đăng nhập B4: Hệ thống cấp token và chuyển vào trang chính theo vai trò B5: Kết thúc
Luồng thay thế	B3.1: Nếu sai mật khẩu, thông báo lỗi và quay lại B2
Luồng ngoại lệ	Nếu hệ thống OAuth (Google) không phản hồi, thông báo lỗi và cho phép đăng nhập nội bộ thay thế

Bảng 6. Đặc tả uc đăng nhập

UC-02: Quản lý các phiên giao dịch và ví

Mục	Nội dung
Mô tả	Cho phép các actor (Cá nhân, Admin) tạo/sửa/xóa các phiên giao dịch hệ thống
Actor chính	Người dùng (Cá nhân, Gia đình)
Actor phụ	Không
Tiền điều kiện	Đã đăng nhập; đã có ít nhất một tài khoản/ví
Hậu điều kiện	Giao dịch mới được lưu vào hệ thống.
Luồng hoạt động chính	B1: Người dùng chọn thêm/sửa/xóa các giao dịch B2: Người dùng chọn ví cần giao dịch, loại giao dịch (income, expense), danh mục và ngày giờ giao dịch B3: Người dùng ghi thêm ghi chú cho phiên giao dịch đó (nếu có) B4: Người dùng nhấn xác nhận B5: Kết thúc
Luồng thay thế	Không có
Luồng ngoại lệ	B2.1: Người dùng không nhập đầy đủ thông tin bắt buộc (ví, số tiền, danh mục hoặc loại giao dịch). Hệ thống hiển thị thông báo lỗi và yêu cầu bổ sung trước khi lưu

Mục	Nội dung
	B2.2: Người dùng nhập số tiền nhỏ hơn hoặc bằng 0. Hệ thống thông báo "Số tiền phải lớn hơn 0" và không lưu giao dịch B2.3: Trong quá trình lưu hoặc cập nhật giao dịch xảy ra lỗi hệ thống hoặc lỗi cơ sở dữ liệu. Hệ thống thông báo thất bại và yêu cầu người dùng thực hiện lại.

Bảng 7. Đặc tả uc quản lý các phiên giao dịch và ví

UC-03: Nhập dữ liệu CSV

Mục	Nội dung
Mô tả	Cho phép người dùng import sao kê ngân hàng (CSV) hoặc ảnh hóa đơn; hệ thống tự động phân loại giao dịch.
Actor chính	Cá nhân
Actor phụ	không
Tiền điều kiện	Đã đăng nhập; đã có ít nhất một tài khoản/ví
Hậu điều kiện	Giao dịch mới được lưu vào hệ thống, đã gán danh mục hoặc ở trạng thái chờ phân loại
Luồng hoạt động chính	B1: Người dùng chọn tài khoản và tải lên file CSV hoặc ảnh hóa đơn B2: Hệ thống đọc, chuẩn hóa dữ liệu và ánh xạ cột (nếu là CSV) B3: Hệ thống kiểm tra và loại bỏ giao dịch trùng lặp B4: Phân loại tự động giao dịch theo quy tắc B5: Hệ thống lưu giao dịch và hiển thị kết quả import B6: Kết thúc
Luồng thay thế	B2.1: Nếu định dạng cột không khớp, yêu cầu người dùng ánh xạ cột thủ công
Luồng ngoại lệ	Nếu file lỗi định dạng hoặc dung lượng vượt giới hạn, thông báo lỗi và hủy import

Bảng 8. Đặc tả uc nhập dữ liệu CSV, hình ảnh

UC-04: Thiết lập ngân sách

Mục	Nội dung
Mô tả	Cho phép người dùng đặt hạn mức chi tiêu theo danh mục theo tháng; hệ thống cảnh báo khi vượt.
Actor chính	Cá nhân
Actor phụ	Hệ thống thông báo (email)
Tiền điều kiện	Đã đăng nhập; đã có danh mục chi tiêu

Mục	Nội dung
Hậu điều kiện	Ngân sách được lưu và bắt đầu theo dõi trong kỳ hiện tại
Luồng hoạt động chính	B1: Người dùng chọn danh mục và nhập hạn mức, kỳ áp dụng B2: Hệ thống lưu ngân sách và bắt đầu đối chiếu chi tiêu thực tế B3: Kết thúc
Luồng thay thế	Khi chi tiêu đạt ngưỡng cảnh báo → Cảnh báo vượt ngân sách: hệ thống gửi thông báo qua Hệ thống thông báo (email)
Luồng ngoại lệ	Nếu hạn mức nhập không hợp lệ (≤ 0), thông báo lỗi và yêu cầu nhập lại

Bảng 9. Đặc tả uc thiết lập ngân sách

UC-05: Theo dõi các chi tiêu

Mục	Nội dung
Mô tả	Hiển thị biểu đồ, số liệu chi tiêu và nhận xét thói quen chi tiêu do AI phân tích.
Actor chính	Cá nhân
Actor phụ	Hệ thống AI nhận xét
Tiền điều kiện	Đã có dữ liệu giao dịch trong hệ thống
Hậu điều kiện	Người dùng xem được xu hướng, cơ cấu chi tiêu và nhận xét từ AI
Luồng hoạt động chính	B1: Người dùng chọn khoảng thời gian cần xem B2: Hệ thống tổng hợp dữ liệu giao dịch theo danh mục và thời gian B3: Hệ thống gửi dữ liệu tổng hợp (có thể cho hệ thống AI để sinh nhận xét) B4: Hệ thống hiển thị biểu đồ B5: Kết thúc
Luồng thay thế	B3.1: Nếu AI không phản hồi kịp thời, hiển thị biểu đồ mà không kèm nhận xét
Luồng ngoại lệ	Nếu không có dữ liệu trong kỳ đã chọn, thông báo "chưa có dữ liệu"

Bảng 10. Đặc tả uc theo dõi chi tiêu

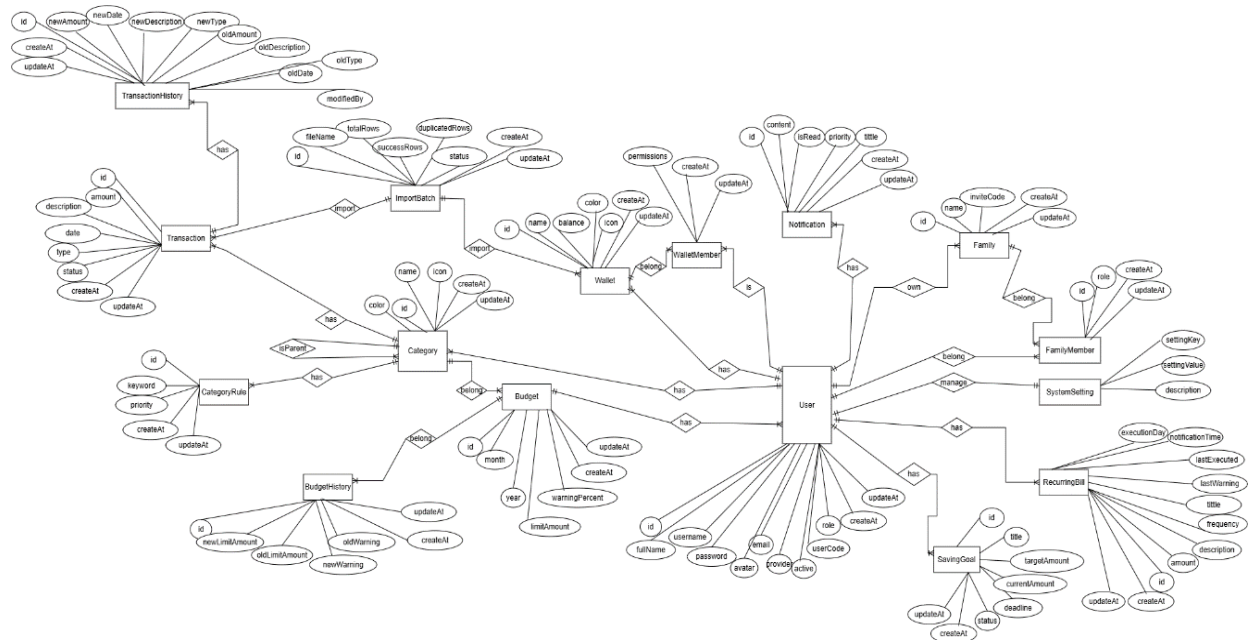
UC-06: Chia sẻ ví chung

Mục	Nội dung
Mô tả	Cho phép chủ tài khoản mời thành viên gia đình cùng theo dõi chi tiêu chung.

Mục	Nội dung
Actor chính	Cá nhân
Actor phụ	Gia đình (thành viên được mời)
Tiền điều kiện	Người mời là chủ sở hữu tài khoản
Hậu điều kiện	Thành viên được cấp quyền truy cập tài khoản theo vai trò đã chọn
Luồng hoạt động chính	B1: Chủ tài khoản nhập email thành viên và chọn quyền (chỉnh sửa/chỉ xem) B2: Hệ thống gửi lời mời cho thành viên B3: Thành viên chấp nhận lời mời B4: Hệ thống cấp quyền truy cập tài khoản B5: Kết thúc
Luồng thay thế	B3.1: Nếu thành viên từ chối, hủy lời mời
Luồng ngoại lệ	Nếu email không tồn tại trong hệ thống, thông báo lỗi

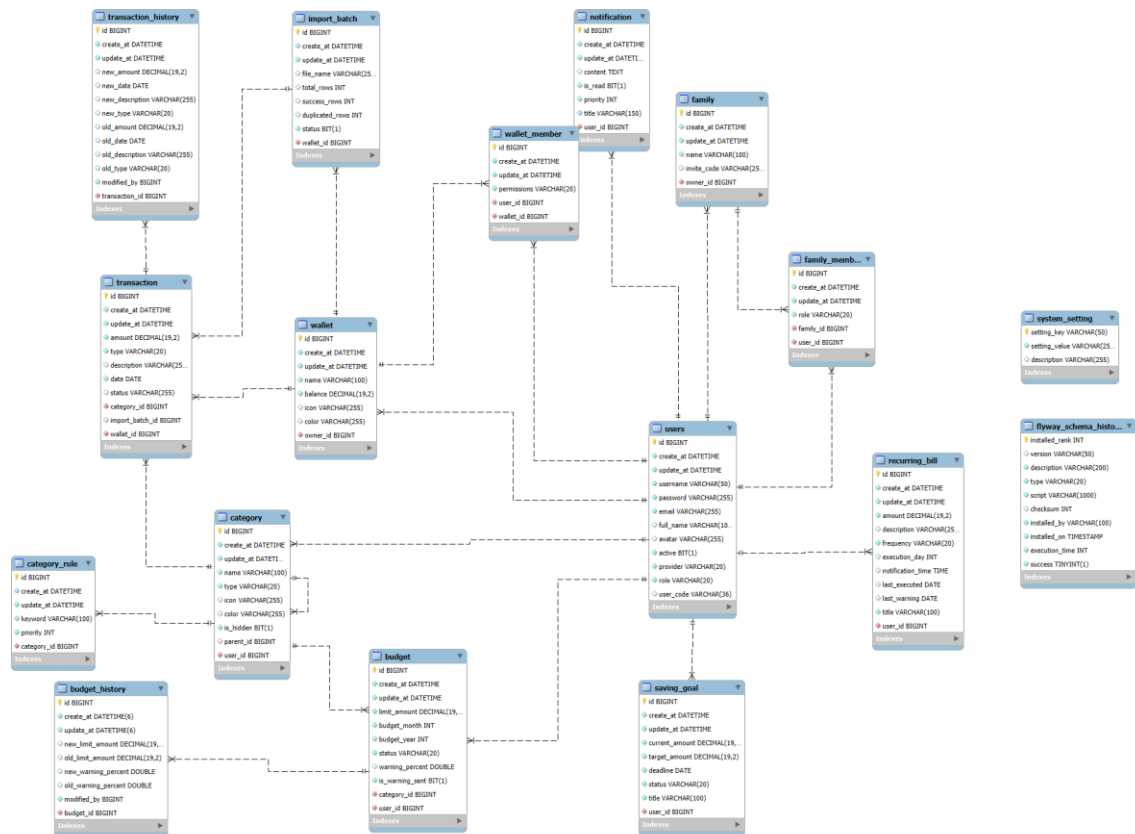
Bảng 11. Đặc tả uc chia sẻ tài khoản

3.2.3. Xây dựng cơ sở dữ liệu hệ thống



Hình 2. Bảng erd của hệ thống

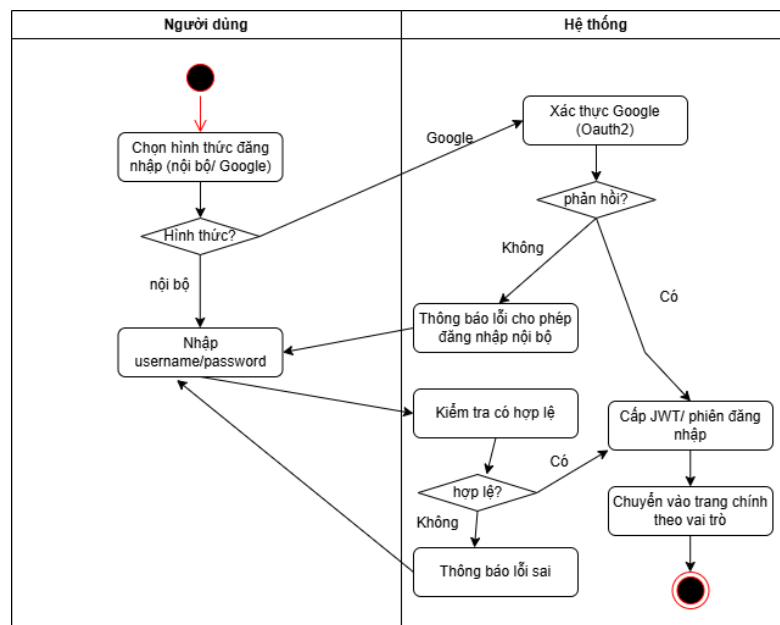
3.2.4. Sơ đồ lớp hệ thống



Hình 3. Sơ đồ lớp của hệ thống

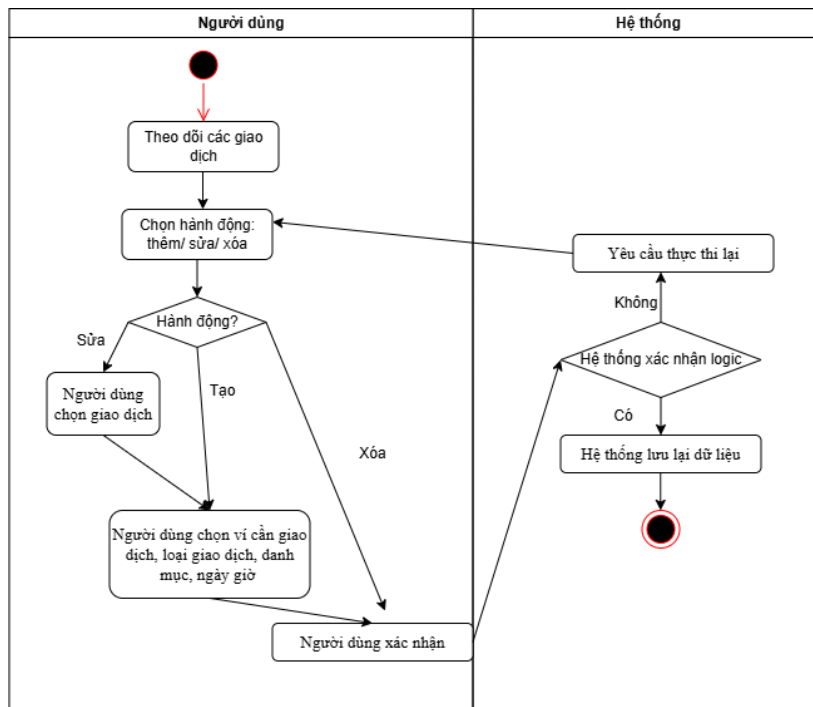
3.2.5. Sơ đồ hoạt động

UC-01: Đăng nhập



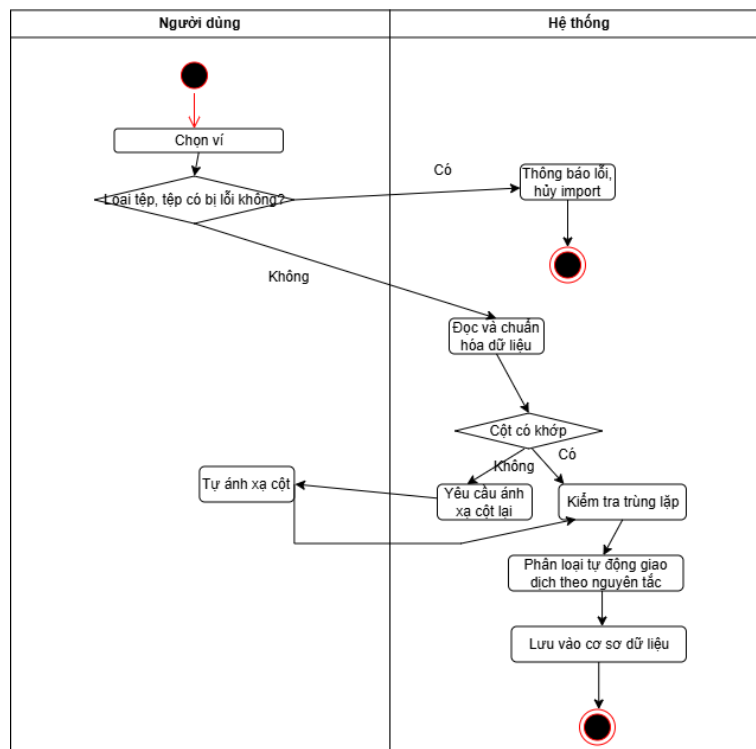
Hình 4. Sơ đồ hoạt động uc đăng nhập

UC-02: Quản lý các phiên giao dịch



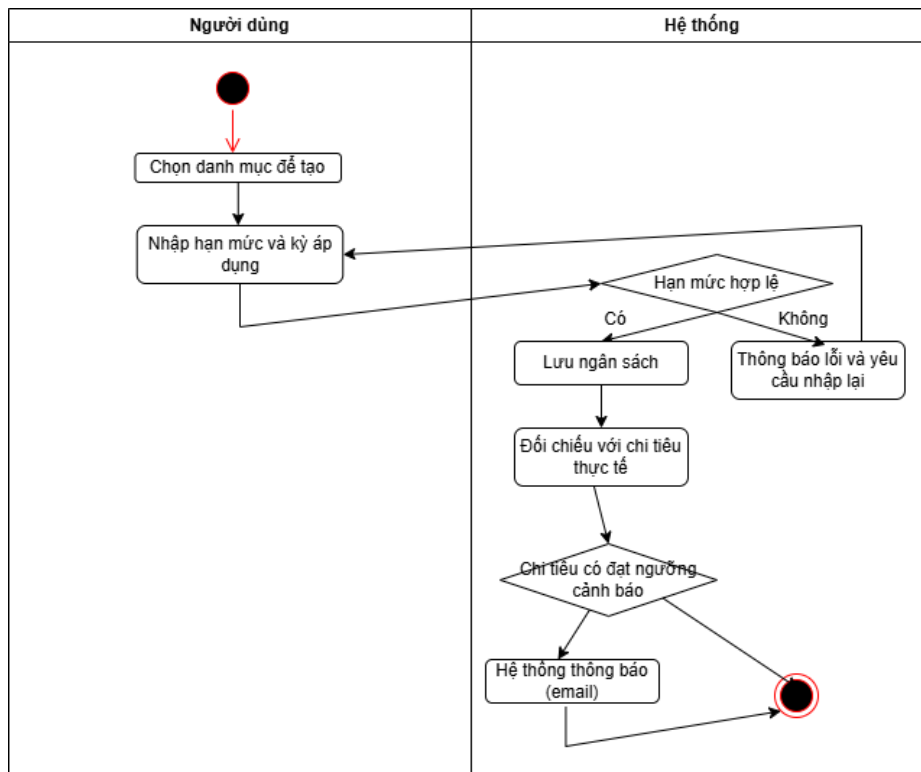
Hình 5 Sơ đồ hoạt động uc quản lý các phiên giao dịch

UC-03: Nhập bằng file dữ liệu



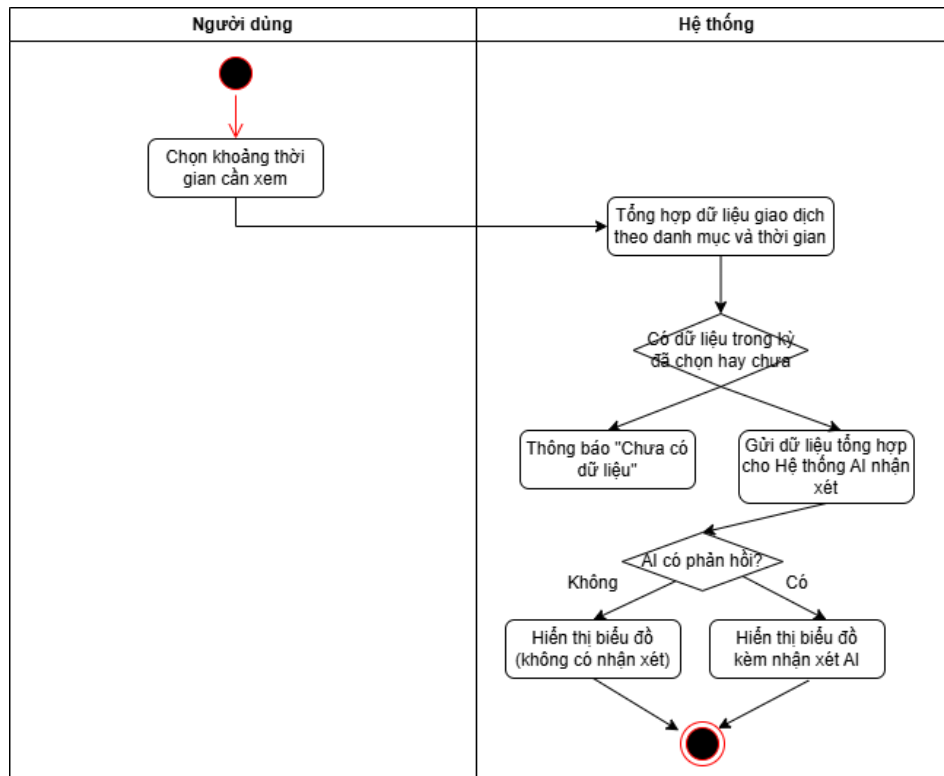
Hình 6. Sơ đồ hoạt động uc nhập dữ liệu CSV, hình ảnh

UC-04: Thiết lập ngân sách



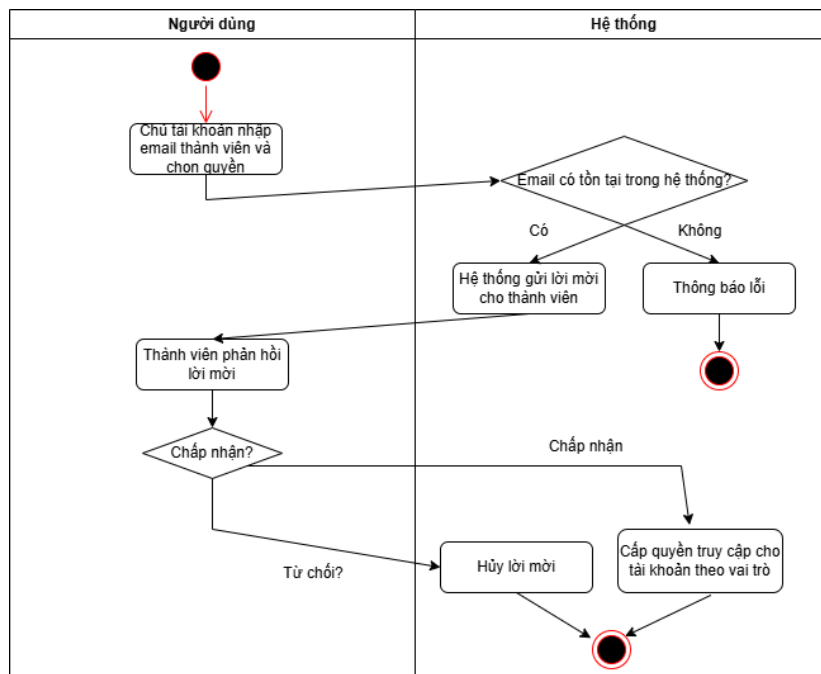
Hình 7. Sơ đồ hoạt động uc thiết lập ngân sách

UC-05: Theo dõi các chỉ tiêu



Hình 8. Sơ đồ hoạt động uc theo dõi các chỉ tiêu

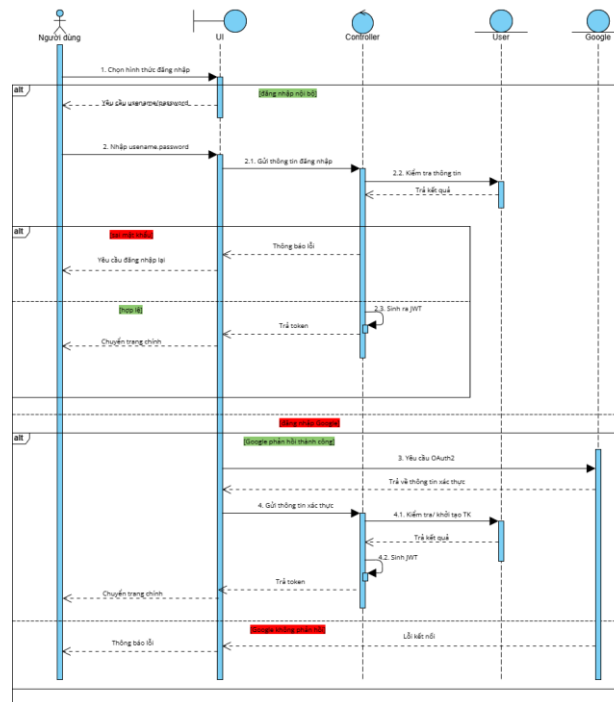
UC-06: Chia sẻ tài khoản



Hình 9. Sơ đồ hoạt động uc chia sẻ tài khoản

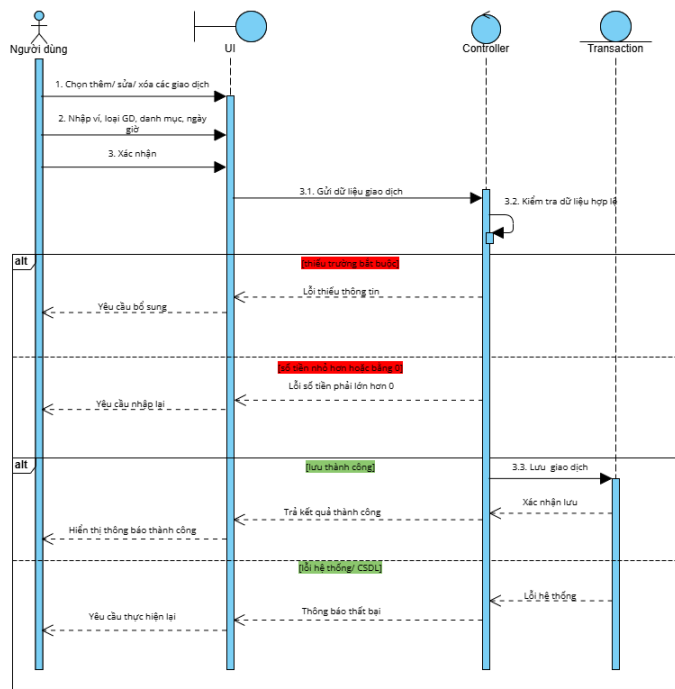
3.2.6. Sơ đồ tuần tự

UC-01: Đăng nhập



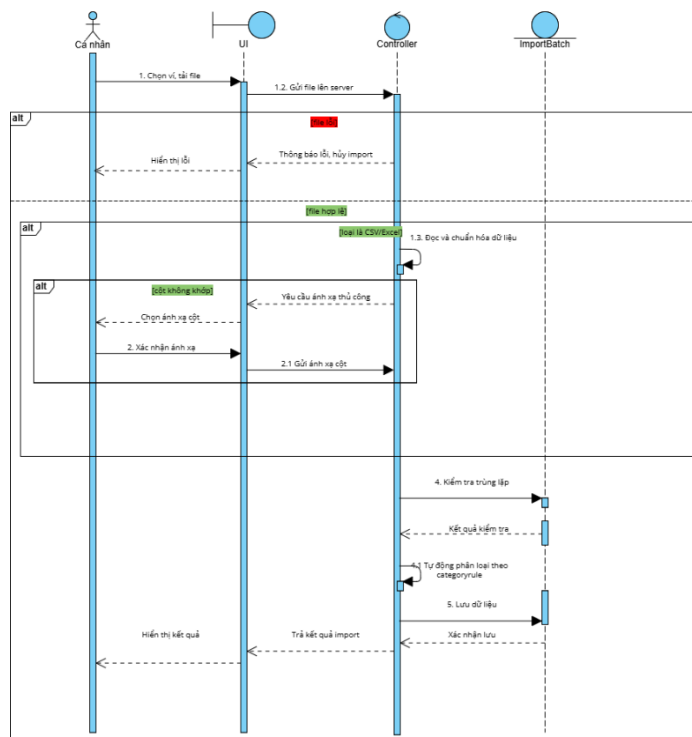
Hình 10. Sơ đồ tuần tự uc đăng nhập

UC-02: Quản lý các phiên giao dịch



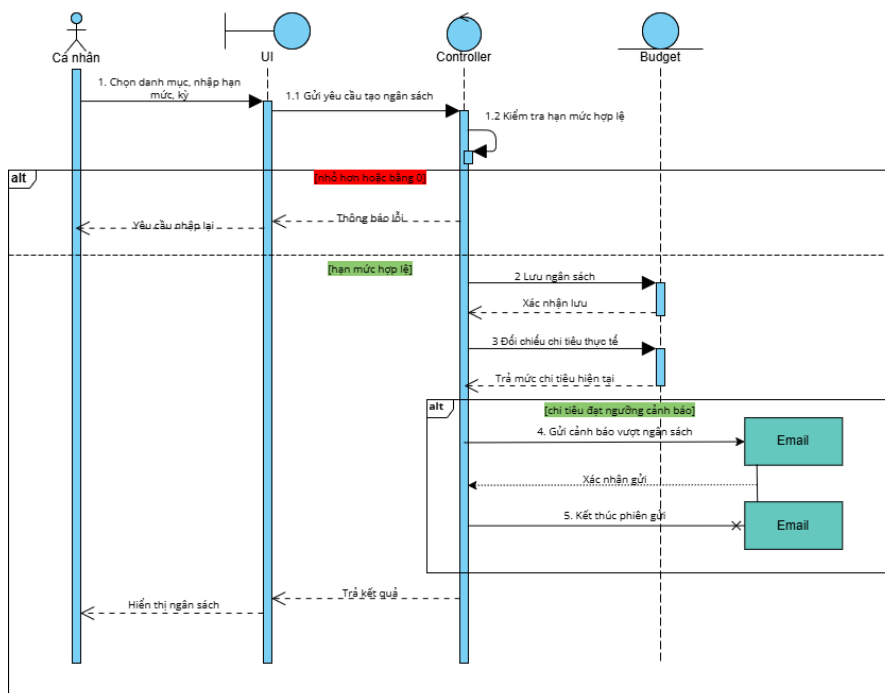
Hình 11. Sơ đồ tuần tự uc quản lý các phiên giao dịch

UC-03: Nhập dữ liệu CSV



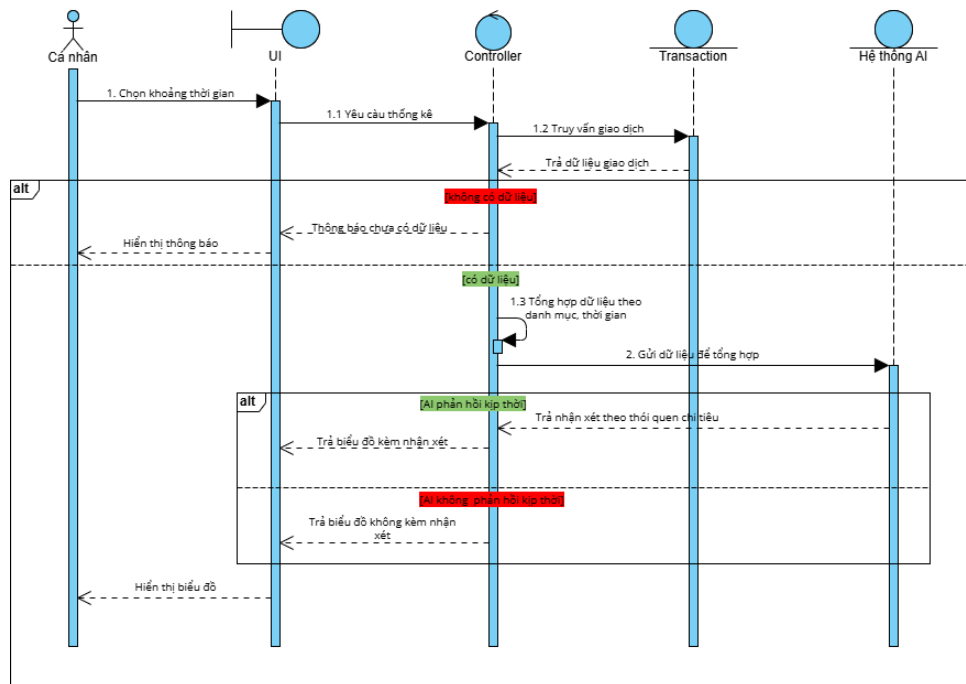
Hình 12. Sơ đồ tuần tự uc nhập dữ liệu CSV, hình ảnh

UC-04: Thiết lập ngân sách



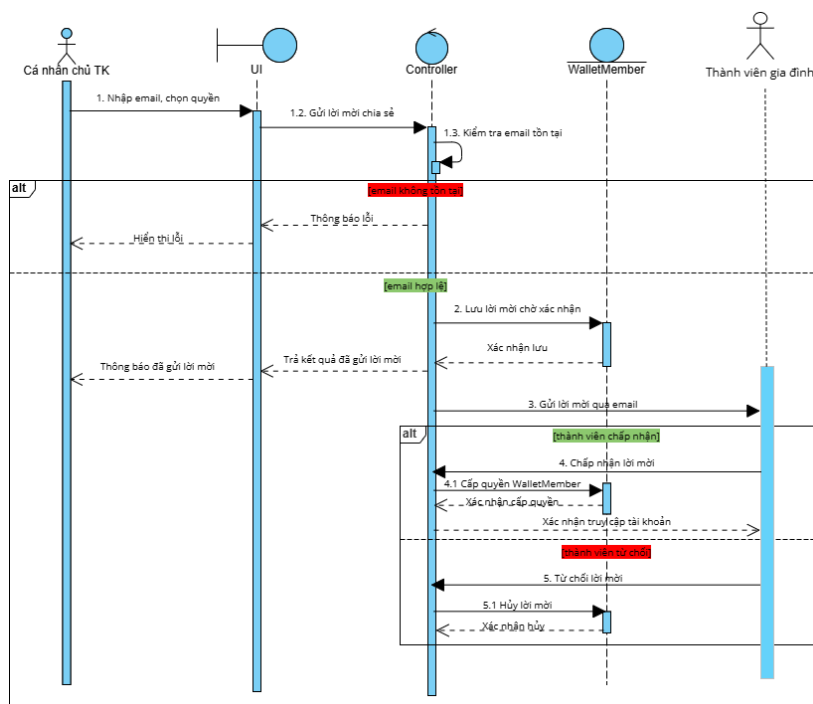
Hình 13. Sơ đồ tuần tự uc thiết lập ngân sách

UC-05: Theo dõi các chi tiêu



Hình 14. Sơ đồ tuần tự uc theo dõi các chi tiêu

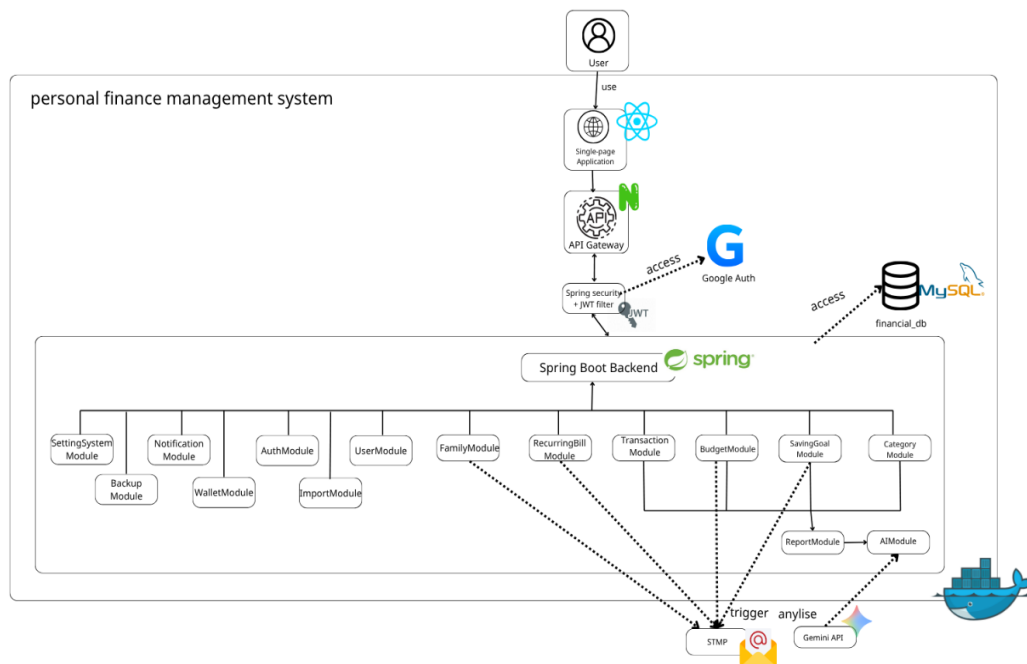
UC-06: Chia sẻ tài khoản



Hình 15. Sơ đồ tuần tự uc chia sẻ tài khoản

3.3 Kiến trúc hệ thống

Hệ thống được xây dựng theo kiến trúc Monolithic: toàn bộ nghiệp vụ được đóng gói trong một ứng dụng Spring Boot duy nhất, nhưng bên trong được chia thành các module độc lập theo từng nghiệp vụ, giúp mã nguồn dễ đọc, dễ bảo trì và sau này có thể dễ dàng tách thành microservices nếu hệ thống mở rộng quy mô. Lựa chọn này phù hợp với NFR-04 (kiến trúc phân tầng, dễ mở rộng) đồng thời tránh độ phức tạp vận hành không cần thiết.



Hình 16: Sơ đồ kiến trúc hệ thống

Các tầng trong kiến trúc:

Thành phần	Công nghệ	Vai trò
Single-page Application	ReactJS + TypeScript	Giao diện người dùng, gọi API qua HTTPS
API Gateway	Nginx	Reverse proxy, phục vụ file tĩnh của SPA, định tuyến api vào Spring Boot Backend
Spring Security + JWT filter	Spring Security, JWT	Xác thực và phân quyền mọi request trước khi vào tầng nghiệp vụ
Spring Boot Backend	Spring Boot, Spring Data JPA	Xử lý toàn bộ logic nghiệp vụ, chia thành 9 module theo domain
MySQL (financial_db)	MySQL	Lưu trữ dữ liệu quan hệ

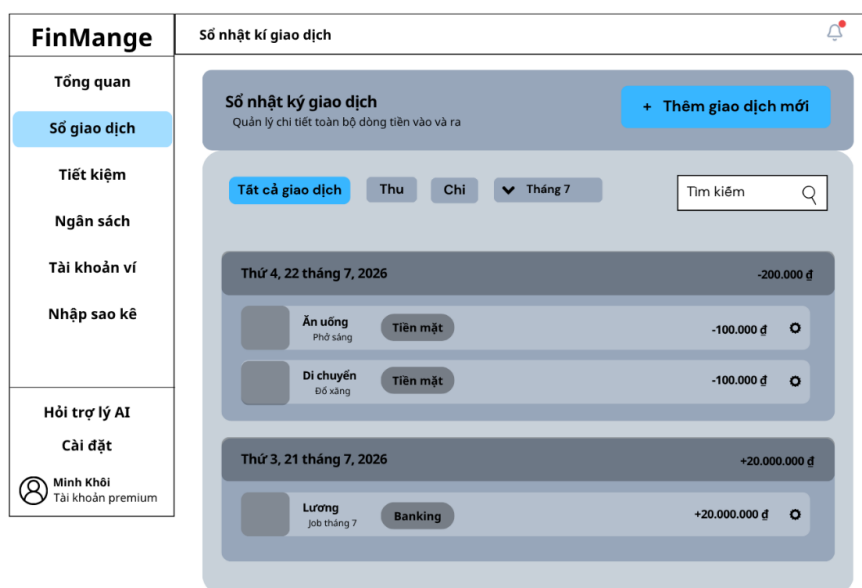
Thành phần	Công nghệ	Vai trò
Google Auth	OAuth2	Xác thực đăng nhập qua tài khoản Google
SMTP	JavaMailSender	Gửi email cảnh báo vượt ngân sách, lời mời chia sẻ tài khoản
GEMINI API	REST API	Sinh các nhận xét chi tiêu từ dữ liệu tổng hợp
Docker	Docker / Docker Compose	Đóng gói và triển khai toàn bộ hệ thống nhất quán giữa môi trường dev và demo

Bảng 12. Bảng các tầng kiến trúc của hệ thống

3.3. Thiết kế giao diện



Hình 17. Wireframe trang tổng quan về tài chính



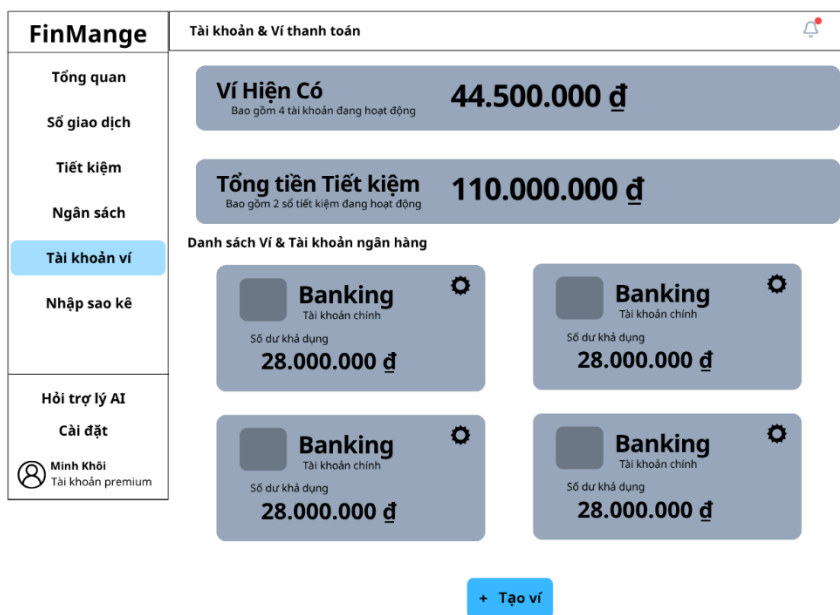
Hình 18. Wireframe số giao dịch



Hình 19. Wireframe số tiết kiệm



Hình 20. Wireframe trang ngân sách



Hình 21. Wireframe trang tài khoản ví

Chương 4. CÀI ĐẶT & TRIỂN KHAI

4.1. Môi trường và công nghệ

Hệ thống xây dựng theo kiến trúc client-server, tách biệt giữa backend (cung cấp API) và frontend (giao diện người dùng). Bảng dưới đây liệt kê các công nghệ chính được sử dụng khi xây dựng và phát triển hệ thống, dựa trên cấu hình thực tế của dự án (pom.xml và package.json).

Backend

Thành phần	Công nghệ	Phiên bản	Vai trò trong đề tài
Ngôn ngữ	Java	17	Ngôn ngữ lập trình chính cho backend
Nền tảng	Spring Boot	3.5.16	Xây dựng REST API, quản lý vòng đời ứng dụng
ORM	Spring Data JPA / Hibernate	6.6.53	Ánh xạ đối tượng – quan hệ, thao tác cơ sở dữ liệu
Cơ sở dữ liệu	MySQL (driver)	9.7.0	Lưu trữ dữ liệu quan hệ của toàn hệ thống
Quản lý migration	Flyway	11.7.2	Quản lý phiên bản schema cơ sở dữ liệu
Bảo mật	Spring Security + JWT	0.12.6	Xác thực, phân quyền theo vai trò (USER/ADMIN)
Đăng nhập ngoài	Google API Client	2.2.0	Đăng nhập bằng tài khoản Google (OAuth2)
Sinh mã tự động	Lombok	1.18.46	Giảm boilerplate code (getter/setter, constructor)
Gửi email	Spring Mail (JavaMailSender)	4.1.0	Gửi thông báo, cảnh báo ngân sách qua email

Thành phần	Công nghệ	Phiên bản	Vai trò trong đề tài
Xuất báo cáo	Apache POI, OpenPDF	5.2.3 / 1.3.30	Xuất báo cáo giao dịch định dạng Excel và PDF
Lưu trữ ảnh	Cloudinary	1.36.0	Lưu ảnh đại diện người dùng
Trí tuệ nhân tạo	Google Gemini API	gemini-3.1-flash-lite	Phân tích, tư vấn tài chính, tự động phân loại giao dịch
Tài liệu API	SpringDoc OpenAPI	2.8.17	Sinh giao diện Swagger UI để kiểm thử API
Kiểm thử	JUnit 5, Mockito, H2	5.12.2 / 5.17.0	Unit test và kiểm thử tích hợp trên CSDL trong bộ nhớ

Bảng 13: Các công nghệ và phiên bản của backend

Frontend

Thành phần	Công nghệ	Phiên bản	Vai trò trong đề tài
Thư viện UI	React	19.2.7	Xây dựng giao diện người dùng dạng SPA
Công cụ build	Vite	8.1.1	Đóng gói và chạy môi trường phát triển
Ngôn ngữ	TypeScript	6.0.2	Kiểm tra kiểu dữ liệu tĩnh, giảm lỗi runtime
Điều hướng	React Router DOM	7.18.1	Định tuyến các trang trong ứng dụng
Giao diện	React Bootstrap + Bootstrap	2.10.10 / 5.3.8	Bộ khung giao diện responsive
Biểu đồ	Recharts	3.10.1	Vẽ biểu đồ tròn chi tiêu, biểu đồ xu hướng thu chi
Gọi API	Axios	1.18.1	Giao tiếp HTTP với backend, có interceptor xử lý token

Thành phần	Công nghệ	Phiên bản	Vai trò trong đề tài
Đăng nhập Google	@react-oauth/google	0.13.5	Tích hợp nút đăng nhập Google phía client

Bảng 14: Các công nghệ và phiên bản của frontend

4.2. Cấu trúc mã nguồn

Mã nguồn của hệ thống được phân chia thành hai thành phần chính gồm backend và frontend. Backend được xây dựng bằng Spring Boot, chịu trách nhiệm xử lý nghiệp vụ, xác thực, phân quyền, truy xuất dữ liệu và cung cấp REST API. Frontend được xây dựng bằng React và TypeScript, chịu trách nhiệm xây dựng giao diện, quản lý trạng thái hiển thị và giao tiếp với backend thông qua HTTP API.

src/

- | — backend/src/main/java/com/personal/finance/backend/
 - | | — users/ — Đăng ký, đăng nhập, hồ sơ, quản trị người dùng
 - | | — wallets/ — Quản lý ví, chia sẻ ví trong gia đình
 - | | — families/ — Nhóm gia đình, mời thành viên
 - | | — transactions/ — Sổ giao dịch, lịch sử chỉnh sửa
 - | | — categories/ — Danh mục thu/chi, quy tắc tự động phân loại
 - | | — budgets/ — Ngân sách chi tiêu theo tháng, cảnh báo vượt mức
 - | | — savingGoals/ — Mục tiêu tiết kiệm
 - | | — bills/ — Hóa đơn định kỳ, nhắc nhở thanh toán
 - | | — notifications/ — Thông báo hệ thống (ưu tiên, đã đọc)
 - | | — reports/ — Tổng hợp dashboard, xuất Excel/PDF
 - | | — importBatch/ — Nhập sao kê ngân hàng (CSV/Excel)
 - | | — ai_assistant/ — Tích hợp Gemini: tư vấn, phân loại tự động
 - | | — backup/ — Sao lưu/phục hồi dữ liệu (JSON)
 - | | — settings/ — Cấu hình hệ thống (do Admin quản lý)
 - | | — security/ — JwtUtil, JwtAuthenticationFilter, SecurityConfig
 - | | — common/ — Base entity, exception handler, scheduler, email
 - | | — configs/ — DataSeeder, cấu hình Cloudinary, OpenAPI
- | — frontend/src/
 - | | — components/ — Component tái sử dụng, chia theo nghiệp vụ
 - | | — pages/ — Các trang tương ứng với route
 - | | — services/ — Lớp gọi API (axios) cho từng nghiệp vụ

- └─ context/ — AuthContext quản lý phiên đăng nhập
- └─ utils/ — Hàm tiện ích (định dạng tiền tệ...)

Bảng 15: Cấu trúc thư mục của mã nguồn

Mỗi package nghiệp vụ ở backend được chia thành bốn tầng gồm controller, service, repository và entity/dto. Controller tiếp nhận request và trả về response, service nhằm xử lý các nghiệp vụ và quy tắc của hệ thống, repository truy xuất dữ liệu thông qua Spring Data JPA, còn entity/dto dùng để định nghĩa và truyền dữ liệu. Cách tổ chức này giúp các module có cấu trúc thống nhất và dễ theo dõi khi phát triển hoặc bảo trì.

4.3. Cài đặt cơ sở dữ liệu

Từ mô hình ERD được thiết kế ở Chương 3, cơ sở dữ liệu được khởi tạo bằng script Flyway V1__Init_schema.sql. Flyway tự động thực hiện script khi hệ thống khởi động, giúp đảm bảo cấu trúc cơ sở dữ liệu thống nhất giữa môi trường phát triển, kiểm thử và triển khai.

Đối chiếu giữa các thực thể trong ERD và các bảng dữ liệu thực tế trong cơ sở dữ liệu:

Thực thể (ERD)	Bảng trong CSDL	Khóa chính	Quan hệ chính	Ghi chú
Người dùng	users	id	1-n với wallet, category, budget	Có user_code (UUID) dùng làm định danh công khai thay vì lộ id tự tăng
Ví	wallet	id	n-1 với users (owner), 1-n với wallet_member	
Thành viên ví chia sẻ	wallet_member	id	n-1 với wallet, users	Ràng buộc UNIQUE(wallet_id, user_id) chống chia sẻ trùng
Gia đình	family	id	1-1 với users (owner)	
Thành viên gia đình	family_member	id	n-1 với family, 1-1 với users	
Danh mục	category	id	tự tham chiếu	Hỗ trợ danh mục cha-con

Thực thể (ERD)	Bảng trong CSDL	Khóa chính	Quan hệ chính	Ghi chú
Quy tắc phân loại	category_rule	id	n-1 với category	Dùng khi nhập sao kê tự động
Giao dịch	transaction	id	n-1 với wallet, category, import_batch	
Lịch sử giao dịch	transaction_history	id	n-1 với transaction	Ghi lại giá trị cũ hoặc mới mỗi lần sửa
Ngân sách	budget	id	n-1 với users, category	UNIQUE(user, category, tháng, năm)
Lịch sử ngân sách	budget_history	id	n-1 với budget	
Mục tiêu tiết kiệm	saving_goal	id	n-1 với users	
Thông báo	notification	id	n-1 với users	
Hóa đơn định kỳ	Recurring_bill	id	n-1 với users	
Cấu hình hệ thống	System_setting	Setting_key		Khóa-giá trị, ví dụ ngưỡng cảnh báo ngân sách mặc định

Bảng 16: Bảng đối chiếu giữa thực thể trong ERD và bảng dữ liệu thật

Hệ thống có cung cấp DataSeeder triển khai CommandLineRunner. Khi ứng dụng khởi động, lớp này sẽ kiểm tra dữ liệu hiện có rồi sau đó tạo các mẫu cần thiết.

Các dữ liệu mẫu bao gồm: tài khoản quản trị, tài khoản người dùng, danh mục hệ thống, ví mẫu, các giao dịch mẫu, ngân sách, mục tiêu tiết kiệm, hóa đơn định kỳ, cấu hình hệ thống. Các dữ liệu mẫu đảm bảo đủ để phục vụ cho tài khoản người dùng mẫu có thể tự trải nghiệm kiểm thử các chức năng.

4.4 Xây dựng các chức năng chính

4.4.1 Nhóm chức năng CRUD cơ bản (đại diện: Quản lý ví)

Chức năng tương ứng với FR-02 (quản lý các giao dịch và ví),, đồng thời tuân thủ nghiêm ngặt quy tắc nghiệp vụ BR-02(mọi thay đổi số dư phải thông qua giao dịch), đặc tả trong WalletController và WalletServiceImpl

Đầu vào (Tạo mới): Tên ví, số dư ban đầu, icon, màu sắc.

Đầu vào (Cập nhật): Tên ví, icon, màu sắc (Hệ thống chủ động khóa trường số dư).

Đầu ra: Thông tin WalletDTO (khi thành công) hoặc mã lỗi 403 Forbidden / 400 Bad Request.

Luồng xử lý: Giao diện (WalletModal.tsx) gửi request qua walletService.ts => WalletController nhận request, đọc userId đã được JwtAuthenticationFilter gắn vào thuộc tính request => WalletServiceImpl.createWallet() kiểm tra người dùng tồn tại, khởi tạo entity Wallet với số dư mặc định bằng 0 nếu không truyền, gán owner => WalletRepository.save() ghi xuống bảng wallet.

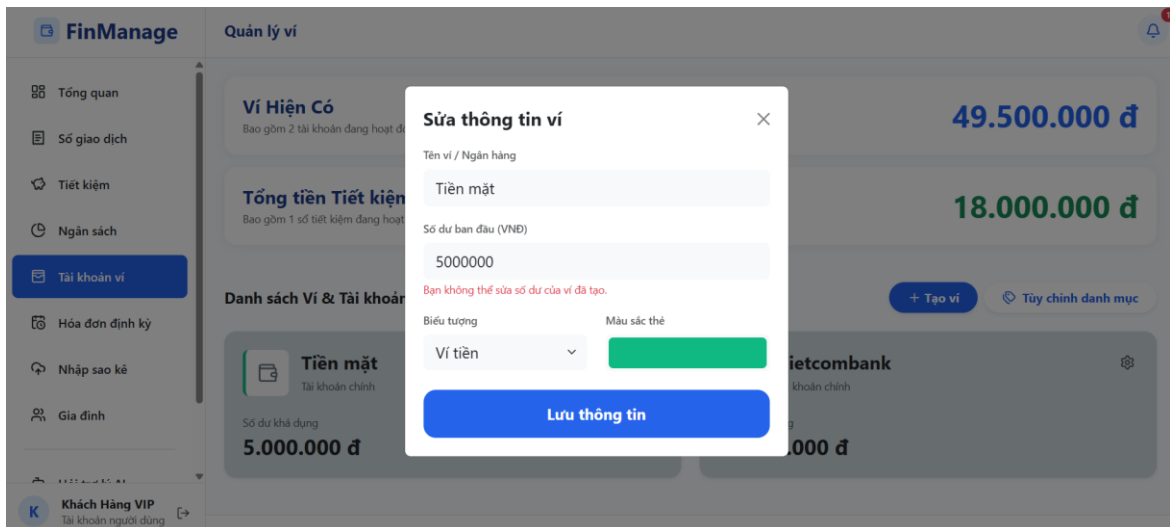
Đoạn mã then chốt: thao tác cập nhật cố tình không cho phép sửa trường balance, buộc mọi thay đổi số dư phải đi qua giao dịch (Transaction) để đảm bảo tính nhất quán của lịch sử tài chính - đây là một quy tắc nghiệp vụ (BR-2) được hình thức hóa trực tiếp trong code.

```
@Override
@Transactional
public WalletDTO updateWallet(Long walletId, Long requesterId,
UpdateWalletRequest request) {
    Wallet wallet = getWalletEntity(walletId);
    requireOwner(wallet, requesterId); // Kiểm tra quyền sở hữu

    wallet.setName(request.getName());
    wallet.setIcon(request.getIcon());
    wallet.setColor(request.getColor());
    // KHÔNG CỐ lệnh wallet.setBalance(request.getBalance());
    return walletMapper.toDTO(walletRepository.save(wallet));
}
```

Hình 22: Đoạn mã then chốt nhóm 1

Bảng chứng chạy:



Hình 23: Bảng chứng chạy nhóm 1

Luồng xử lý tương tự cũng được áp dụng cho quản lý danh mục (CategoryService) và mục tiêu tiết kiệm (SavingGoalService), trong đó mỗi nghiệp vụ có thêm các quy tắc đặc thù riêng. Ví dụ, danh mục không được phép xóa nếu đã phát sinh giao dịch liên quan; thay vào đó, danh mục sẽ được chuyển sang trạng thái "ẩn" để đảm bảo không ảnh hưởng đến dữ liệu giao dịch và lịch sử tài chính.

4.4.2. Ghi nhận giao dịch và tính nhất quán dữ liệu

Đây là chức năng lõi vì mọi thao tác tài chính khác như ngân sách, mục tiêu tiết kiệm và báo cáo đều phụ thuộc vào dữ liệu giao dịch. Tương ứng với FR-02, xử lý đồng thời các quy tắc nghiệp vụ BR-01 (bắt buộc phải có danh mục và ví), BR-02 (tác động số dư), BR-06 (quyền truy cập ví) và BR-08 (lưu vết lịch sử).

Đầu vào: walletId, categoryId, amount, type (INCOME/EXPENSE), date, description.

Đầu ra: HTTP 201 Created kèm TransactionDTO, hoặc HTTP 403 nếu cố tình thêm giao dịch vào ví người khác được chia sẻ với quyền "Chỉ xem".

Luồng xử lý: Khi có yêu cầu tạo giao dịch, hệ thống trước tiên kiểm tra quyền sửa ví (hasEditPermission). Sau đó, giao dịch được lưu vào bảng transaction. Trong cùng một transaction của cơ sở dữ liệu, hệ thống cập nhật số dư ví và kiểm tra ngân sách. Khi sửa hoặc xóa giao dịch, hệ thống sẽ hoàn lại phần thay đổi của giao dịch cũ, cập nhật số dư theo dữ liệu mới và lưu lịch sử thay đổi vào bảng transaction_history.

Đoạn mã then chốt: mỗi giao dịch khi được tạo, sửa hoặc xóa đều phải đồng thời cập nhật số dư của ví trong cùng một transaction cơ sở dữ liệu, nhằm đảm bảo số dư và lịch sử giao dịch luôn nhất quán, tránh xảy ra tình trạng dữ liệu bị lệch nhau.

```
// TransactionServiceImpl.java
@Override
@Transactional
public TransactionDTO createTransaction(Long userId, CreateTransactionRequest request) {
    boolean canEditWallet = walletRepository.hasEditPermission
        (request.getWalletId(), userId);
    if (!canEditWallet) {
        throw new AccessDeniedException("Bạn không có quyền thêm giao dịch vào ví này!");
    }
    ...
    Transaction savedTransaction = transactionRepository.save(transaction);
    // Xác định mức độ ảnh hưởng số dư: Thu nhập cộng vào, Chi tiêu trừ đi
    BigDecimal deltaAmount = request.getType() == Transaction.TransactionType.INCOME
        ? request.getAmount()
        : request.getAmount().negate();
    // Thực thi Native Query: UPDATE Wallet w SET w.balance = w.balance + :amount
    walletRepository.updateBalance(wallet.getId(), deltaAmount);
    // Kích hoạt cơ chế kiểm tra ngân sách (Liên kết FR-11)
    if (request.getType() == Transaction.TransactionType.EXPENSE) {
        budgetService.checkAndAlertBudget(userId, request.getCategoryId(),
            request.getDate().getMonthValue(), request.getDate().getYear());
    }
    return transactionMapper.toDTO(savedTransaction);
}
```

Hình 24: Đoạn mã then chốt nhóm 2

Ba điểm đáng lưu ý trong đoạn mã trên: (1) kiểm tra quyền EDIT trên ví trước khi ghi nhận giao dịch, phục vụ tính năng chia sẻ ví trong gia đình; (2) `walletRepository.updateBalance()` sử dụng câu lệnh `UPDATE ... SET balance = balance + :amount` thay vì thực hiện theo cách đọc–sửa–ghi, giúp hạn chế lỗi race condition khi có nhiều giao dịch được thực hiện đồng thời; (3) sau khi ghi nhận khoản chi, hệ thống gọi ngay `budgetService.checkAndAlertBudget()` để kiểm tra ngân sách trong tháng có bị vượt hoặc sắp vượt hay không, liên kết trực tiếp với FR-11.

Khi sửa hoặc xóa giao dịch, nghiệp vụ tương tự được áp dụng theo chiều ngược lại: hệ thống hoàn tác ảnh hưởng của giá trị cũ lên số dư trước khi áp dụng giá trị mới. Đồng thời, một bản ghi `TransactionHistory` được lưu lại nếu có bất kỳ trường nào thay đổi, phục vụ chức năng xem lịch sử đối soát trên giao diện.

Bảng chứng chạy:

Sửa giao dịch

Loại giao dịch

Khoản thu (+)Khoản chi (-)

Số tiền (VNĐ)

500000

Danh mục

Ăn uống

Ví thanh toán

Ví Sinh Hoạt Tháng 9 - Số dư: 1.500.000đ

Ngày giao dịch

09/13/2026

Ghi chú

Tùy chọn...

Lưu giao dịch

Ăn uống

Tháng 9/2026

0 đ

Còn lại: 7.000.000 đ

/ 7.000.000 đ

Ăn uống

Tháng 9/2026

500.000 đ

Còn lại: 6.500.000 đ

/ 7.000.000 đ

Hình 25: Bảng chứng chạy nhóm 2

4.4.3. Ngân sách chi và cảnh báo tự động

Hệ thống hiện thực FR-09 (Tạo ngân sách) và FR-11 (Cảnh báo), tuân thủ các quy tắc BR-04 (kiểm tra theo kỳ), BR-05 (cấu hình ngưỡng) và BR-10 (ưu tiên cảnh báo).

Đầu vào: categoryId, month, year, limitAmount, warningPercent (tùy chọn).

Đầu ra: Cảnh báo in-app và gửi Email trực tiếp đến người dùng khi đạt ngưỡng.

Luồng xử lý: Hệ thống áp dụng ràng buộc UNIQUE(user, category, month, year) để tránh tạo trùng ngân sách. Hàm checkAndAlertBudget được tự động gọi khi có giao dịch chi tiêu. Hàm này tính tổng chi tiêu thực tế trong tháng và so sánh với hai ngưỡng: (1) Ngưỡng vượt mức 100% và (2) Ngưỡng cảnh báo do người dùng thiết lập hoặc mặc định là 80%.

Đoạn mã then chốt:

Cơ chế cảnh báo (checkAndAlertBudget) được kích hoạt tự động mỗi khi có giao dịch chi tiêu mới thuộc danh mục đang được đặt ngân sách. Hệ thống tính tổng chi tiêu trong tháng và so sánh với hai ngưỡng: ngưỡng cảnh báo (mặc định 80%, có thể tùy chỉnh thông qua warningPercent của từng ngân sách hoặc lấy từ cấu hình hệ thống DEFAULT_BUDGET_WARNING_PERCENT) và ngưỡng vượt mức (100%). Mỗi mức cảnh báo chỉ được gửi một lần, sử dụng cờ warningSent và trạng thái EXCEED, nhằm tránh gửi thông báo lặp lại cho người dùng ở các giao dịch tiếp

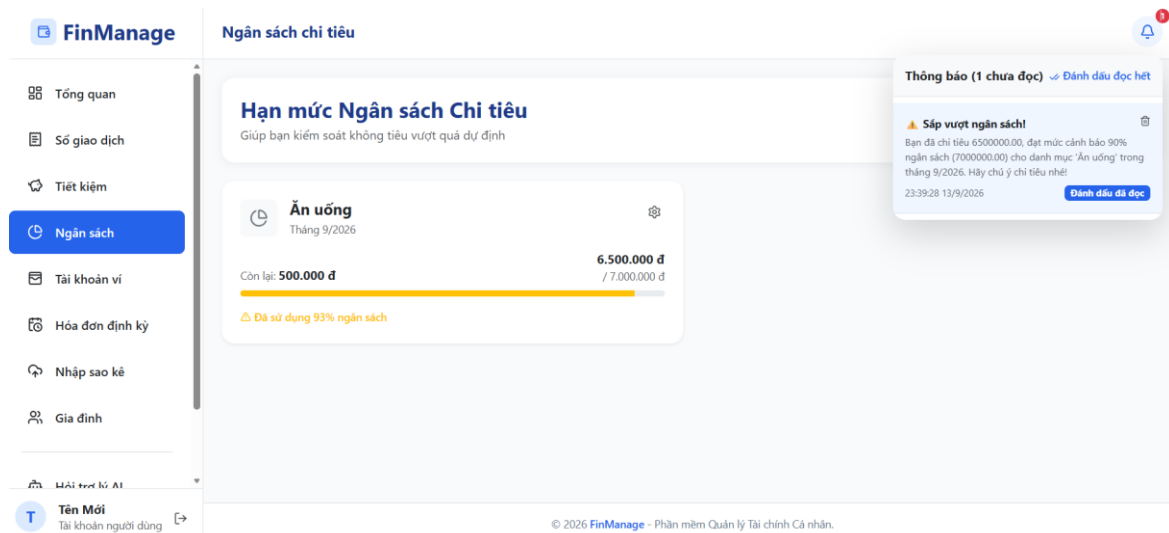
theo. Thông báo được gửi đồng thời qua hai kênh: bảng notification trong ứng dụng và email thông qua SMTP.

```
BigDecimal totalSpent = transactionRepository.sumExpenseByCategoryAndMonth(categoryId, userId, month, year, TransactionType.EXPENSE);
BigDecimal warningLimit = budget.getLimitAmount().multiply(BigDecimal.valueOf(budget.getWarningPercent() / 100.0));

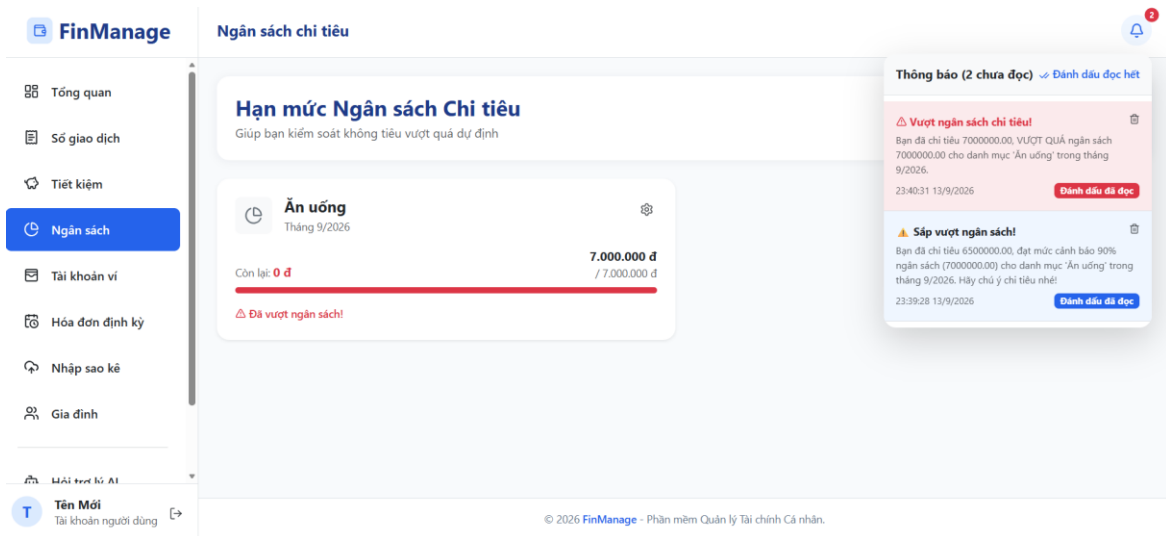
// Vượt 100% (Mức ưu tiên 1)
if (totalSpent.compareTo(budget.getLimitAmount()) >= 0 && budget.getStatus() != BudgetStatus.EXCEED) {
    budget.setStatus(BudgetStatus.EXCEED);
    notificationService.createSystemNotification(userId, "Vượt ngân sách!", msg, 1);
    emailService.sendEmail(user.getEmail(), "Cảnh báo vượt ngân sách", msg);
}
// Chạm ngưỡng cảnh báo (Mức ưu tiên 2) và đảm bảo chỉ gửi 1 lần (BR-10)
else if (totalSpent.compareTo(warningLimit) >= 0 && totalSpent.compareTo(budget.getLimitAmount()) < 0 && !budget.isWarningSent()) {
    budget.setWarningSent(true);
    notificationService.createSystemNotification(userId, "Sắp vượt ngân sách!", msg, 2);
}
```

Hình 26: Đoạn mã then chốt nhóm 3

Bảng chứng chạy:



Hình 27: Bảng chứng chạy nhóm 3, đến % quy định



Hình 28: Bằng chứng chạy nhóm 3, vượt mức

4.4.4. Nhập sao kê giao dịch và chuẩn hóa dữ liệu (Khử trùng lặp)

Gắn mã: Mục này hiện thực chức năng Nhập dữ liệu sao kê, tương ứng FR-06, FR-07 và tuân thủ quy tắc nghiệp vụ BR-03 (Phát hiện và khử giao dịch trùng lặp).

Đầu vào: Tập sao kê (.csv, .xlsx), các tham số ánh xạ cột (Ngày, Số tiền, Nội dung).

Đầu ra: Giao dịch được nạp vào cơ sở dữ liệu, đối tượng ImportBatchDTO thống kê số dòng thành công, thất bại và trùng lặp.

Luồng xử lý: gồm hai bước bước xem trước (POST /import-batches/preview) đọc dòng tiêu đề và trả về danh sách cột cho frontend để người dùng tự lựa chọn cột; bước nhập chính thức (POST /import-batches/upload) xử lý toàn bộ dữ liệu được tải lên. Với mỗi dòng dữ liệu, hệ thống kiểm tra trùng lặp dựa trên ví, ngày, số tiền và nội dung để tránh nhập lại giao dịch đã tồn tại theo BR-03.

Hệ thống xử lý file theo cấu hình cột đã được ánh xạ, đồng thời chuẩn hóa định dạng ngày tháng do dữ liệu từ các ngân hàng có thể sử dụng các định dạng khác nhau như dd/MM/yyyy hoặc yyyy-MM-dd.

Đoạn mã then chốt:

```
// 1. Chuẩn hóa định dạng ngày tháng linh hoạt từ file CSV/Excel
//của nhiều ngân hàng khác nhau
private LocalDate parseFlexibleDate(String dateStr) {
    String[] dateFormats = {"yyyy-MM-dd", "dd/MM/yyyy", "d/M/yyyy", "MM/dd/yyyy", "dd-MM-yyyy"};
    for (String format : dateFormats) {
        try { return LocalDate.parse(dateStr, DateTimeFormatter.ofPattern(format)); }
        catch (Exception ignored) {}
    }
    throw new RuntimeException("Không hỗ trợ định dạng ngày: " + dateStr);
}

// 2. Vòng lặp xử lý dữ liệu và Khử trùng lặp (BR-03)
for (String[] fields : rawData) {
    LocalDate date = parseFlexibleDate(fields[dateCol]);
    Double rawAmount = Double.parseDouble(fields[amountCol]);
    BigDecimal absoluteAmount = BigDecimal.valueOf(Math.abs(rawAmount));
    String descStr = fields[descCol];

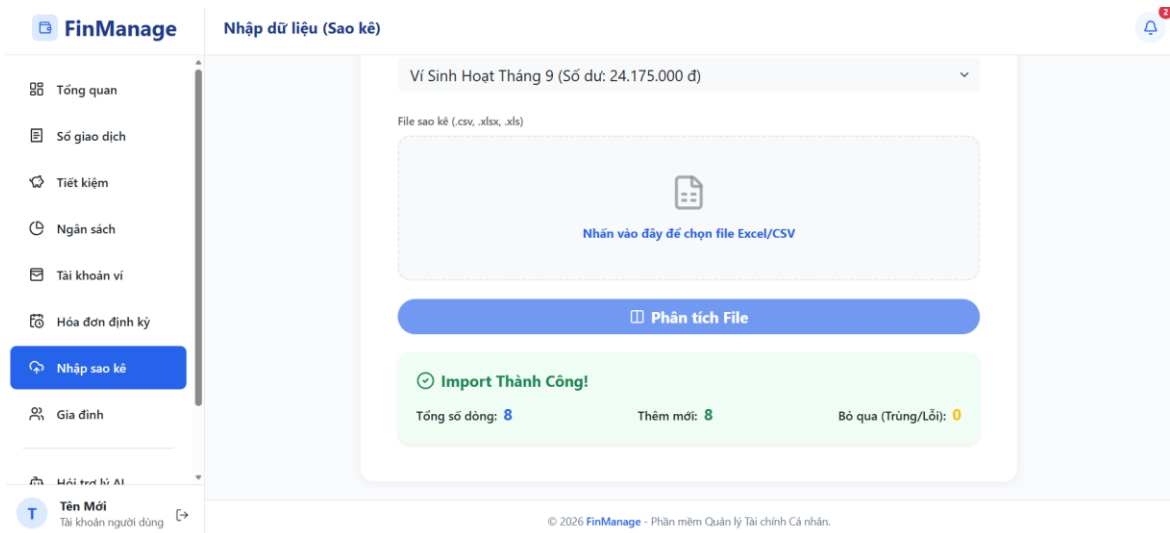
    // Truy vấn CSDL để bỏ qua các dòng đã từng được import trước đó
    if (transactionRepository.existsByWalletIdAndDateAndAmountAndDescription(
        walletId, date, absoluteAmount.doubleValue(), descStr)) {
        duplicateRows++;
        continue; // Bỏ qua, không lưu dòng này
    }

    // ... Khởi tạo Transaction và chuyển tiếp cho module Phân loại (Xem Mục 4.5) ...
    successRows++;
}
}
```

Hình 29: Đoạn mã then chốt 4

Bảng chứng chạy:

Hình 30: Bảng chứng chạy nhóm 4, định dạng cột



Hình 31: Bằng chứng chạy nhóm 4, khử trùng lặp

4.4.5. Chia sẻ ví gia đình và phân quyền truy cập

Hiện thực chức năng chia sẻ tài khoản (FR-13), xử lý quyền truy cập giữa các tài khoản (BR-06) và kiểm tra tính hợp lệ của người được mời (BR-12).

Đầu vào: email người được mời, quyền permission (VIEW / EDIT).

Đầu ra: HTTP 200 (Thêm thành viên thành công, gửi Email) hoặc HTTP 400 (Từ chối nếu Email không hợp lệ).

Luồng xử lý:

Khi chủ ví gửi yêu cầu chia sẻ, hệ thống kiểm tra Email có tồn tại trong bảng Users hay không (BR-12). Sau đó, hệ thống kiểm tra người mời và người được mời có thuộc cùng một Family hay không. Nếu hợp lệ, hệ thống tạo bản ghi trong bảng wallet_member và gửi email thông báo đến người được mời.

Đoạn mã then chốt:

Đoạn mã thể hiện sự kiểm soát logic nghiệp vụ chặt chẽ, từ chối việc tạo bản ghi rác (pending invite) nếu email chưa đăng ký hệ thống.

```

// BR-12: Chặn ngay nếu Email không tồn tại trong CSDL
User targetUser = userRepository.findByEmail(request.getEmail())
    .orElseThrow(() -> new RuntimeException("Không tìm thấy người dùng với email này"));

if (wallet.getOwner().getId().equals(targetUser.getId())) {
    throw new RuntimeException("Không thể mời chính chủ sở hữu vào ví!");
}

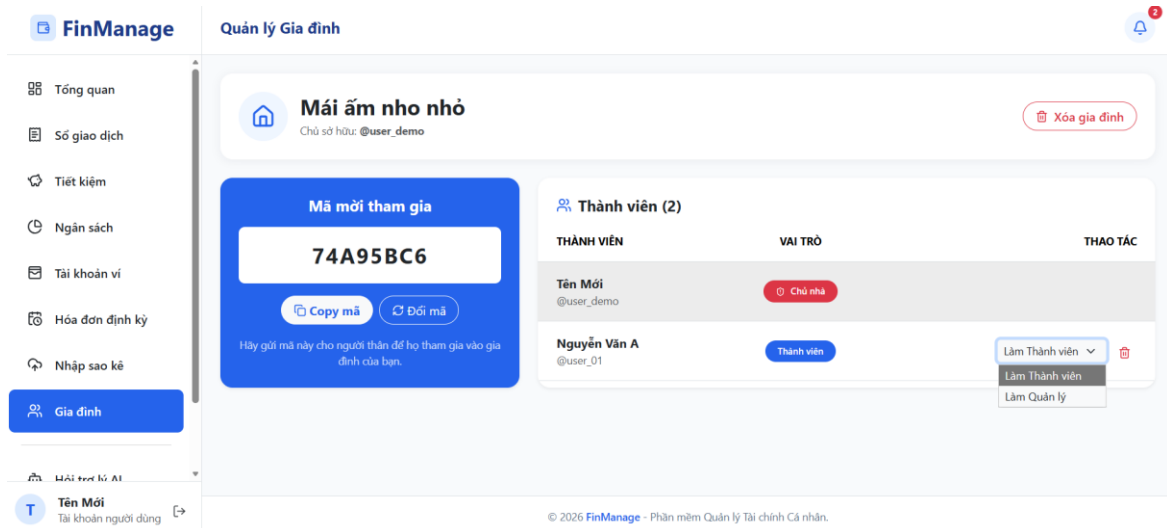
// Ràng buộc bảo mật: Chỉ cho phép chia sẻ nội bộ trong Family
if (ownerFamily == null || targetFamily == null || !ownerFamily.getId().
equals(targetFamily.getId())) {
    throw new RuntimeException("Chỉ có thể chia sẻ ví cho các thành viên trong cùng một (
}

// Cấp quyền và thông báo
WalletMember member = new WalletMember();
member.setWallet(wallet);
member.setUser(targetUser);
member.setPermissions(request.getPermission());
walletMemberRepository.save(member);
emailService.sendEmail(targetUser.getEmail(), "Bạn được mời tham gia quản lý ví!",
    emailBody);

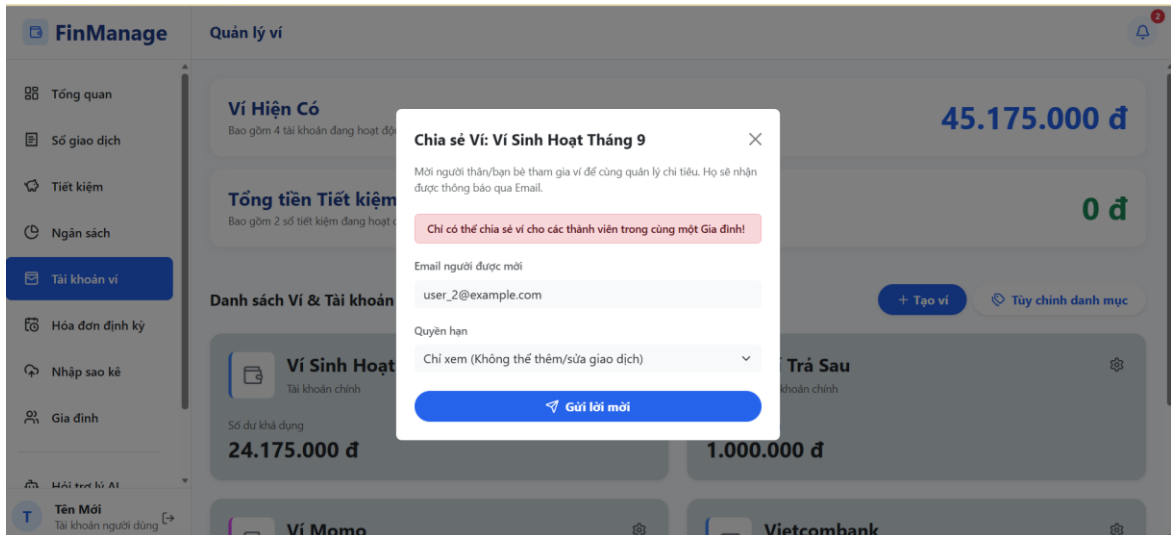
```

Hình 32: Đoạn mã then chốt nhóm 5

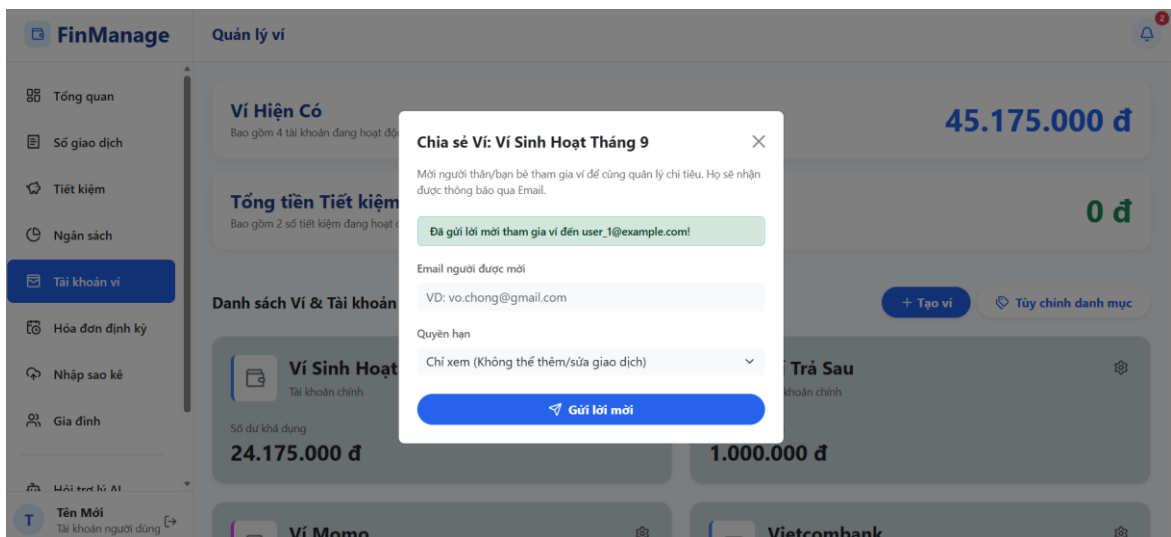
Bảng chứng chạy:



Hình 33: Bảng chứng chạy nhóm 5, minh họa gia đình



Hình 34: Bằng chứng chạy nhóm 5, mời khác thành viên



Hình 35: Bằng chứng chạy nhóm 5, mời thành viên gia đình

4.5. Phần lỗi kỹ thuật đặc thù

Bài toán khó nhất trong đề tài không nằm ở việc lưu trữ dữ liệu tài chính (vốn là CRUD tương đối chuẩn mực), mà nằm ở việc tự động gán danh mục cho một giao dịch chỉ dựa trên một dòng mô tả ngắn, không có cấu trúc, xuất hiện ở hai nơi: nhập sao kê hàng loạt bằng file Excel hoặc CSV.

Vì sao không dùng một mô hình học máy tự huấn luyện: với một ứng dụng cá nhân, mỗi người dùng có tập giao dịch riêng, không đủ dữ liệu để huấn luyện một mô hình phân loại văn bản có độ chính xác chấp nhận được; đồng thời việc huấn luyện và duy trì mô hình vượt quá phạm vi và thời lượng của một đề án. Hệ thống cân nhắc ba

phương án: (1) chỉ dùng luật từ khóa thuần túy - đơn giản, nhanh, nhưng không xử lý được các nội dung giao dịch chưa từng gặp; (2) chỉ dùng AI cho toàn bộ giao dịch - linh hoạt nhưng chi phí và độ trễ cao khi nhập hàng trăm dòng cùng lúc; (3) kết hợp cả hai theo thứ tự ưu tiên, để luật xử lý phần đã biết và AI chỉ can thiệp vào phần chưa biết. Phương án (3) được chọn vì cân bằng giữa tốc độ, chi phí và độ chính xác.

Luồng xử lý:

Bước 1 - Áp dụng luật trước: mỗi Category có thể gắn nhiều CategoryRule (từ khóa, độ ưu tiên). Khi xử lý một mô tả giao dịch, hệ thống duyệt qua danh sách luật theo thứ tự ưu tiên giảm dần, chuyển cả từ khóa và mô tả về chữ thường và kiểm tra quan hệ "chứa".

Bước 2 - Với các giao dịch không khớp với bất kỳ luật nào, hệ thống gom các mô tả thành một mảng và gọi AI một lần duy nhất cho toàn bộ lô dữ liệu. Đồng thời, danh sách toàn bộ danh mục hiện có của người dùng, bao gồm ID và tên danh mục, được truyền vào phần chỉ dẫn hệ thống. AI được yêu cầu trả về kết quả theo đúng định dạng JSON thuần túy {"Nội dung": ID}, giúp backend có thể chuyển đổi trực tiếp kết quả mà không cần thực hiện thêm bước xử lý ngôn ngữ tự nhiên.

Bước 3 - Xử lý khi AI trả lỗi hoặc trả về định dạng không đúng: lời gọi API được bọc trong khối try/catch. Nếu Gemini trả về chuỗi có kèm markdown code fence (json), hệ thống chủ động loại bỏ các ký tự thừa trước khi thực hiện parse. Nếu quá trình parse thất bại hoặc API gặp lỗi mạng, phương thức trả về một Map rỗng thay vì ném ngoại lệ làm gián đoạn toàn bộ quá trình nhập liệu. Các giao dịch không phân loại được vẫn được lưu vào danh mục "Chưa phân loại" để người dùng tự điều chỉnh sau, thay vì làm hỏng toàn bộ file nhập.

Đoạn mã then chốt:

```

// BƯỚC 1: Phân loại theo Luật từ khóa trước (Tối ưu tốc độ)
Category matchedCategory = categorizeTransaction(descStr, userRules, null);

if (matchedCategory != null) {
    transaction.setCategory(matchedCategory);
} else {
    // Gom các giao dịch chưa nhận diện được vào danh sách để gọi AI
    transaction.setCategory(uncategorized);
    needAiCategorization.add(transaction);
    descriptionsForAi.add(descStr);
}

// BƯỚC 2: Gọi AI xử lý theo lô (Batch) để giảm số lượt request API
if (!descriptionsForAi.isEmpty()) {
    log.info("Gọi AI để phân loại {} giao dịch...", descriptionsForAi.size());
    Map<String, Long> aiResults = aiAssistantService
        .categorizeTransactionsBatch(descriptionsForAi, allCategories);

    for (Transaction t : needAiCategorization) {
        Long predictedCategoryId = aiResults.get(t.getDescription());
        if (predictedCategoryId != null) {
            // Gán lại danh mục dựa trên kết quả AI
            allCategories.stream().filter(c -> c.getId().equals(predictedCategoryId))
                .findFirst().ifPresent(t::setCategory);
        }
    }
}

// BƯỚC 3 (Trong AiAssistantServiceImpl): Ép kiểu trả về và xử lý lỗi JSON an toàn
String systemInstruction = "Bạn là trợ lý phân loại chi tiêu... Trả về JSON thuần túy đi";
String responseStr = callGeminiTextApi(systemInstruction, userPrompt);

// Chủ động dọn dẹp các ký tự markdown thừa do AI sinh ra
responseStr = responseStr.replace("`json", "").replace("`", "").trim();
return objectMapper.readValue(responseStr, new TypeReference<Map<String, Long>>(){});

```

Bảng chứng chạy:

FinManage		Sổ giao dịch
Tổng quan	Thứ Bảy, 15 tháng 8, 2025	-200.000 đ
Sổ giao dịch	Chưa phân loại Đang (kế tiếp: Netflix... (Xem Hoạt Động))	-260.000 đ
Tiết kiệm	Thứ Năm, 13 tháng 8, 2025	-1.450.000 đ
Ngân sách	Mua sắm Chi tiêu tự động 1 (Xem chi tiết)	-1.450.000 đ
Tài khoản ví	Thứ Tư, 12 tháng 8, 2025	-65.000 đ
Hóa đơn định kỳ	Ăn uống Highlands Coffee Lan... (Xem Hoạt Động)	-65.000 đ
Nhập sao kê	Thứ Hai, 10 tháng 8, 2025	-1.500.000 đ
Gia đình	Chưa phân loại Thanh toán tín dụng... (Xem Hoạt Động)	-1.500.000 đ
Hỗ trợ kỹ thuật	Thứ Tư, 5 tháng 8, 2025	27.600.000 đ
Cài đặt tài khoản	Mua sắm Chi tiêu tự động 31 (Xem chi tiết)	-400.000 đ
Tên Mới Tài khoản người dùng	Chưa phân loại Nhận lương tháng 7... (Xem Hoạt Động)	+28.000.000 đ

Hình 36: Bảng chứng chạy phần lõi kỹ thuật, các giao dịch đã import

A1							
	A	B	C	D	E	F	G
1	Ngày giao dịch	So tien	Nội dung giao dịch				
2	5/8/2026	28000000	Nhan lương tháng 7 công ty TNHH ABC				
3	10/8/2026	-1500000	Thanh toán tiền điện nước tháng 7				
4	12/8/2026	-65000	Highlands Coffee Landmark 81				
5	12/9/2026	5000000	Thuong hoàn thành dự án quý 3				
6	15/8/2026	-260000	Dang ký gói Netflix Premium				
7	20/8/2026	-500000	Đo xăng xe máy và rửa xe				
8	21/8/2026	-1200000	Mua sam quan áo trên Shopee				
9	22/8/2026	-300000	An toi nhà hàng lẩu Haidilao				
10							
11							
12							
13							

Hình 37: Bảng chứng chạy phần lõi kỹ thuật, ảnh excel import

4.6. Đóng gói và chuyển khai

Hệ thống được đóng gói hoàn toàn bằng Docker và Docker Compose, đảm bảo tính nhất quán tuyệt đối giữa môi trường phát triển (máy cá nhân) và môi trường triển khai thực tế (máy chủ production). Cấu trúc triển khai bao gồm ba dịch vụ (services) chạy song song: cơ sở dữ liệu MySQL, Spring Boot Backend và ReactJS Frontend.

Để chạy lại toàn bộ hệ thống trên một máy tính mới chỉ cần cài đặt Docker, các bước thực hiện được tối ưu hóa trong 4 lệnh thao tác trên terminal:

Lệnh 1: git clone <https://github.com/khoile/finmanage.git>

Lệnh 2: cd finmanage

Lệnh 3: git checkout tags/v1.0

Lệnh 4: docker-compose up -d --build

Tên biến	Ý nghĩa
DB_URL, DB_USERNAME, DB_PASSWORD	Thông tin kết nối MySQL
JWT_SECRET, JWT_EXPIRATION_MS	Khóa bí mật và thời hạn token đăng nhập
GOOGLE_CLIENT_ID	Định danh ứng dụng cho đăng nhập Google OAuth2

Tên biến	Ý nghĩa
GEMINI_API_KEY	Khóa gọi Google Gemini API cho tính năng AI
MAIL_USERNAME, MAIL_PASSWORD	Tài khoản SMTP gửi email cảnh báo
VITE_API_BASE_URL	Địa chỉ backend để frontend gọi API

Bảng 17: Bảng về biến môi trường

Các giá trị nhạy cảm (khóa thật, mật khẩu) không được đưa vào báo cáo hoặc commit lên GitHub. Các file `.env.example` và `application-local.yaml.example` chỉ chứa các giá trị mẫu (placeholder).

Migration khi triển khai: Flyway tự động chạy các script trong `src/main/resources/db/migration/` mỗi khi ứng dụng khởi động (`spring.flyway.enabled=true`), giúp schema được đồng bộ với phiên bản code đang chạy mà không cần thao tác thủ công trên môi trường sản xuất.

Chương 5. KIỂM THỬ VÀ ĐÁNH GIÁ

5.1. Chiến lược kiểm thử

Chiến lược kiểm thử của hệ thống tập trung vào việc đảm bảo tính chính xác của các chức năng tài chính và bảo mật dữ liệu. Phạm vi kiểm thử gồm ba mức chính:

Kiểm thử mức đơn vị (Unit Test): Tập trung vào các hàm xử lý logic ở Backend như tính toán số dư ví, kiểm tra ngân sách và logic kết hợp giữa Rule-based và AI trong module `ImportBatch`.

Kiểm thử tích hợp (Integration Test): Kiểm tra API trả về đúng dữ liệu, xử lý các mã lỗi (400, 401, 403, 404) và đảm bảo các Entity được ánh xạ đúng với cơ sở dữ liệu.

Kiểm thử thủ công (Manual E2E Test): Thực hiện trên giao diện ReactJS để kiểm tra giao diện và xác nhận các luồng nghiệp vụ từ đầu đến cuối.

Môi trường kiểm thử được triển khai ở hai môi trường: môi trường phát triển (Local DEV) sử dụng cơ sở dữ liệu H2 In-memory cho các bài kiểm thử và môi trường DEMO chạy trên Docker. Dữ liệu kiểm thử (Test Data) được tạo tự động thông qua

lớp DataSeeder, bao gồm các tài khoản Admin và User cùng dữ liệu mẫu về giao dịch, ví và ngân sách.

Tiêu chí hoàn thành kiểm thử (Exit Criteria) được xác định khi các quy tắc nghiệp vụ từ BR-01 đến BR-12 đều có test case tương ứng và chạy đạt (Pass), hệ thống ngăn chặn được các trường hợp truy cập trái phép giữa các User và quy trình CI (Continuous Integration) biên dịch thành công không phát sinh lỗi.

5.2. Thiết kế testcase

Bộ Test Case được thiết kế bao phủ các luồng cơ bản, các luồng vi phạm quy tắc nghiệp vụ và các luồng lỗi dữ liệu. Để đảm bảo tính súc tích cho báo cáo, dưới đây trích lọc các test case quan trọng nhất. Danh sách đầy đủ test case toàn bộ hệ thống được đính kèm tại tệp test-cases.md với unit test và thư mục docs/kiem-thu/ với integration test.

Nhóm CRUD cơ bản (đại diện là wallet)

Mã TC	Chức năng / UC	Mục tiêu	Tiền điều kiện	Dữ liệu đầu vào	Các bước	Kết quả mong đợi
TC-WALL-001	Tạo Ví	Khởi tạo ví mới với số dư ban đầu hợp lệ.	User tồn tại.	name: "Ví Tiết Kiệm", balance: 100000	Gọi hàm createWallet().	HTTP 201; Wallet được tạo và trả về DTO.
TC-WALL-002	Tạo Ví	Báo lỗi khi ID người dùng không tồn tại.	User không tồn tại trong DB.	userId: 99L	Gọi hàm createWallet() với ID ảo.	HTTP 400; Ném RuntimeException "Không tìm thấy người dùng!".
TC-WALL-003	Sửa Ví	Chủ sở hữu cập nhật thông tin ví thành công.	Ví thuộc sở hữu của User (requesterId = 1L).	walletId: 10L, name: "Ví Mới"	Gọi hàm updateWallet().	HTTP 200; Tên mới được cập nhật vào DB.
TC-WALL-004	Sửa Ví	Ngăn chặn sửa ví nếu không có quyền sở hữu.	Ví thuộc User A, User B thao tác.	walletId: 10L, notOwnerId: 2L	Gọi hàm updateWallet() bởi User B.	HTTP 403; Ném AccessDeniedException "Bạn không có quyền thao tác trên ví này!".

Mã TC	Chức năng / UC	Mục tiêu	Tiền điều kiện	Dữ liệu đầu vào	Các bước	Kết quả mong đợi
TC-WALL-005	Xóa Ví	Chủ sở hữu xóa hoàn toàn dữ liệu ví.	User là Owner của ví.	walletId: 10L, requesterId : 1L	Gọi hàm deleteWallet().	HTTP 204; Thực thi lệnh delete() trên DB.

Bảng 18: Bảng testcase nhóm 1

Ghi nhận giao dịch và tính nhất quán dữ liệu

Mã TC	Chức năng / UC	Mục tiêu	Tiền điều kiện	Dữ liệu đầu vào	Các bước	Kết quả mong đợi
TC-TRAN-003	Xóa GD	Xóa GD CHI thành công và hoàn trả số dư Ví gốc.	Giao dịch tồn tại, User có quyền EDIT.	transactionId: 100L, userId: 10L	Gọi hàm deleteTransaction().	HTTP 204; Xóa Transaction, gọi updateBalance() cộng lại 50.000.
TC-TRAN-004	Xóa GD	Ngăn User xóa giao dịch thuộc quyền quản lý người khác.	Giao dịch không thuộc sở hữu/không access được.	transactionId: 100L, hackerId: 99L	Gọi hàm deleteTransaction().	HTTP 400; Ném RuntimeException "Không tìm thấy giao dịch hoặc truy cập trái phép!".
TC-TRAN-005	Sửa GD	Cập nhật số tiền (CHI 50k - > 70k), bù trừ số dư Ví.	User có quyền EDIT, GD tồn tại.	amount: 70000, type: EXPENSE	Gọi hàm updateTransaction().	HTTP 200; Lưu thay đổi, gọi updateBalance() trừ tiếp 20.000.
TC-TRAN-006	Sửa GD	Đổi loại (CHI 50k -> THU 100k), bù trừ số dư Ví.	User có quyền EDIT, GD tồn tại.	amount: 100000, type: INCOME	Gọi hàm updateTransaction().	HTTP 200; Đổi loại, gọi updateBalance() cộng bù 150.000.
TC-TRAN-007	Sửa GD	Chặn sửa giao dịch khi chỉ có quyền VIEW.	User chỉ có quyền VIEW trên ví chứa giao dịch.	Payload cập nhật giao dịch.	Gọi hàm updateTransaction().	HTTP 403; Ném AccessDeniedException "Bạn không có quyền sửa giao dịch trong ví này!".
TC-TRAN-008	Xóa GD	Chặn xóa giao dịch khi chỉ có quyền VIEW.	User chỉ có quyền VIEW trên ví chứa giao dịch.	transactionId: 100L, userId: 11L	Gọi hàm deleteTransaction().	HTTP 403; Ném AccessDeniedException "Bạn không có quyền xóa giao dịch trong ví này!".

Bảng 19: Bảng testcase nhóm 2

Ngân sách chi và cảnh báo tự động

Mã TC	Chức năng / UC	Mục tiêu	Tiền điều kiện	Dữ liệu đầu vào	Các bước	Kết quả mong đợi
TC-TRA-N-009	Thêm GD	Tự động gọi kiểm tra Ngân sách khi phát sinh khoản CHI.	Giao dịch loại EXPENSE hợp lệ.	type: EXPENSE	Gọi hàm createTransaction().	Giao dịch lưu thành công, tự động trigger budgetService.checkAndAlertBudget().
TC-BUDG-002	Tạo NS	Chặn tạo trùng lặp ngân sách cùng danh mục và kỳ.	Đã tồn tại Budget cho tháng/năm/danh mục đó.	Thông số y hệt TC-BUDG-001	Gọi hàm createBudget() lần 2.	HTTP 400; Ném RuntimeException "Bạn đã thiết lập ngân sách cho danh mục này...".
TC-BUDG-006	Cảnh báo	Vượt 100% ngân sách: Đổi Status, bắn Alert mức 1.	Ngân sách 5M ACTIVE.	Tổng chi thực tế = 5.5M	Gọi hàm checkAndAlertBudget().	Status đổi EXCEED, tạo Noti (Priority 1), gửi Email Khẩn.
TC-BUDG-007	Cảnh báo	Chạm ngưỡng (80%): Đổi cò, bắn Alert mức 2.	Ngân sách 5M, ngưỡng 80%.	Tổng chi thực tế = 4.5M	Gọi hàm checkAndAlertBudget().	Cờ warningSent=true, tạo Noti (Priority 2), gửi Email.
TC-BUDG-008	Cảnh báo	Dưới ngưỡng: Không báo động giả mạo.	Ngân sách 5M, ngưỡng 80%.	Tổng chi thực tế = 2M	Gọi hàm checkAndAlertBudget().	Cờ warningSent=false, KHÔNG sinh Noti, KHÔNG gửi Email.

Bảng 20: Bảng testcase nhóm 3

Nhập sao kê và chuyển hóa dữ liệu

Mã TC	Chức năng / UC	Mục tiêu	Tiền điều kiện	Dữ liệu đầu vào	Các bước	Kết quả mong đợi
TC-IMP-001	Trích xuất	Đọc chuẩn xác các Headers từ cấu trúc file CSV.	Không có	File CSV: "Ngày,Số tiền,Nội dung"	Gọi hàm extractHeaders().	Trả về danh sách 3 cột đúng tên tiêu đề.
TC-IMP-002	Nạp CSV	Chặn Import File vào Ví nếu chỉ có quyền VIEW.	User chỉ có quyền VIEW trên ví đích.	File CSV hợp lệ	Gọi hàm importData().	HTTP 403; Ném AccessDeniedException "Bạn không có quyền import dữ liệu vào ví này!".
TC-IMP-003	Nạp CSV	Import thành công, ưu tiên dùng Rule (Không gọi AI).	Tồn tại Rule "Highlands" - > "Ăn uống".	File CSV chứa dòng "Highlands Coffee"	Gọi hàm importData().	Trả successRows=1. API categorizeTransactionsBatch KHÔNG chạy.
TC-IMP-004	Khử trùng	Bỏ qua các dòng giao dịch bị trùng (Deduplication).	Giao dịch y hệt đã tồn tại trong CSDL.	File CSV chứa dòng giao dịch cũ	Gọi hàm importData().	Trả duplicatedRows=1. Hàm saveAll() KHÔNG thực thi.
TC-IMP-005	Nạp CSV	Bắt lỗi định dạng số tiền/ngày và Rollback tiến trình.	File CSV chứa dữ liệu lỗi định dạng cột ngày.	Cột ngày = "ABC"	Gọi hàm importData().	Ném IllegalArgumentException "Phát hiện dòng sai định dạng số tiền hoặc ngày tháng!".

Bảng 21: Bảng testcase nhóm 4

Chia sẻ ví và phân quyền

Mã TC	Chức năng / UC	Mục tiêu	Tiền điều kiện	Dữ liệu đầu vào	Các bước	Kết quả mong đợi
TC-WALL-006	Chia sẻ Ví	Chia sẻ cho tài khoản cùng nhóm Gia đình.	Cùng chung Family, target user tồn tại.	email: " friend@example.com ", permission: VIEW	Gọi hàm shareWallet().	HTTP 200; Lưu WalletMember, gọi emailService gửi thông báo.

Mã TC	Chức năng / UC	Mục tiêu	Tiền điều kiện	Dữ liệu đầu vào	Các bước	Kết quả mong đợi
TC-WA-LL-007	Chia sẻ Ví	Ràng buộc: Chặn tự chia sẻ quyền cho chính mình.	Email truyền vào trùng email Owner.	email: " owner_user@example.com "	Gọi hàm shareWallet() mời chính mình.	HTTP 400; Ném RuntimeException "Không thể mời chính chủ sở hữu vào ví!".
TC-WA-LL-008	Chia sẻ Ví	Ràng buộc: Chặn chia sẻ cho tài khoản ngoài nhóm.	Email thuộc Family khác.	email: " stranger@example.com "	Gọi hàm shareWallet() với user ngoài nhóm.	HTTP 400; Ném RuntimeException "Chỉ có thể chia sẻ ví cho các thành viên trong cùng một Gia đình!".
TC-FA-MI-001	Tạo nhóm	Khởi tạo nhóm và tự gán quyền Chủ (MANAGER).	User chưa thuộc gia đình nào.	name: "Gia đình mới"	Gọi hàm createFamily().	HTTP 201; Lưu Family, FamilyMember, sinh mã mời UUID.
TC-FA-MI-002	Tạo nhóm	Chặn 1 User tham gia/tạo nhiều gia đình.	User đã là thành viên của gia đình khác.	name: "Nhà thứ 2"	Gọi hàm createFamily().	HTTP 400; Ném RuntimeException "Bạn đã thuộc một gia đình khác...".

Bảng 22: Bảng testcase nhóm 5

5.3. Kiểm thử tự động và CI

Hệ thống sử dụng JUnit 5 để kiểm thử tự động ở tầng Backend. Các module xử lý chính như tính số dư (TransactionService), kiểm tra ngân sách (BudgetService) và xử lý các tác vụ định kỳ (FinanceTaskScheduler) đều được kiểm thử bằng Unit Test.

Các bài kiểm thử tự động được tích hợp vào quy trình CI (Continuous Integration) thông qua GitHub Actions. Mỗi khi lập trình viên thực hiện git push lên nhánh main, GitHub Actions sẽ tự động thiết lập môi trường Java 17, tải các thư viện Maven và chạy lệnh mvn test để kiểm tra hệ thống.

```
Results:

Tests run: 86, Failures: 0, Errors: 0, Skipped: 0

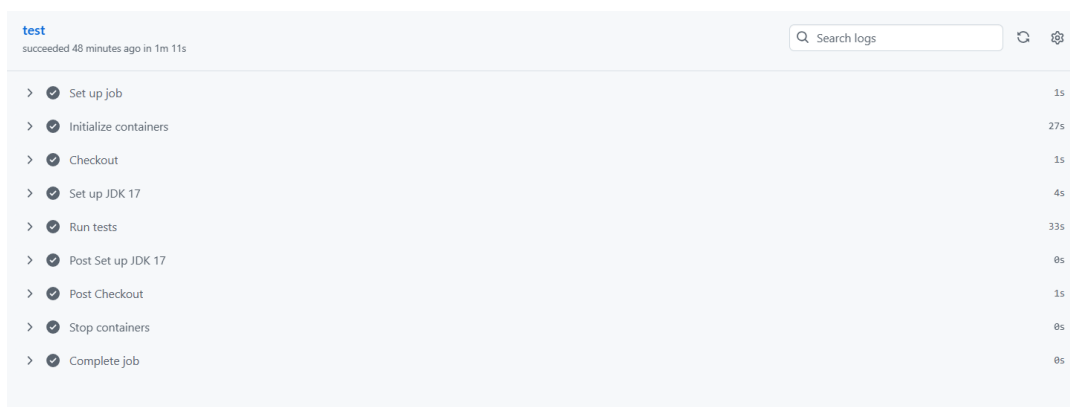
-----

BUILD SUCCESS

-----

Total time: 35.978 s
Finished at: 2026-09-15T00:55:06+07:00

-----
```



Bảng 23: Bảng kiểm thử thành công unit test và CI & CD

5.4. Ma trận truy vết yêu cầu với testcase

Ma trận dưới đây chứng minh mức độ bao phủ của bộ Test Case đối với các Yêu cầu chức năng (FR) và Quy tắc nghiệp vụ (BR) đã được thiết kế tại Chương 3:

Mã yêu cầu	Nội dung	Test case phủ	Kết quả lượt gần nhất
FR-01	Đăng ký, đăng nhập, Google OAuth, JWT	TC-AUTH-001, TC-AUTH-002, TC-AUTH-003, TC-AUTH-004, TC-AUTH-005 (Chưa có Test Case cho Login bằng Google)	Pass
FR-02	Quản lý giao dịch, ví & Xem/Lọc lịch sử	TC-WALL-001 đến TC-WALL-005 TC-TRAN-001 đến TC-TRAN-008	Pass
FR-03	Thống kê thu chi (Biểu đồ, AI advice)	TC-REPO-001, TC-REPO-002	Pass

Mã yêu cầu	Nội dung	Test case phủ	Kết quả lượt gần nhất
		(Chưa có Test Case cho phần nhận xét từ AI)	
FR-04	Sao lưu và phục hồi dữ liệu cá nhân	TC-BACK-001, TC-BACK-002, TC-BACK-003	Pass
FR-05	Tùy biến danh mục (Màu sắc, Icon), tạo ví	TC-WALL-001, TC-CATE-001, TC-CATE-002	Pass
FR-06	Nhập sao kê CSV/Excel.	TC-IMP-001, TC-IMP-002, TC-IMP-005	Pass
FR-07	Chuẩn hóa, phát hiện trùng lặp khi import.	TC-IMP-001, TC-IMP-005	Pass
FR-08	Tự động phân loại bằng Rule/AI.	TC-IMP-003, TC-IMP-004, TC-CATE-006, TC-CATE-007	Pass
FR-09	Thiết lập ngân sách & ghi nhận lịch sử.	TC-BUDG-001 đến TC-BUDG-008	Pass
FR-10	Đặt mục tiêu tiết kiệm, theo dõi tiến độ.	TC-SAVE-001 đến TC-SAVE-007 TC-SCHED-004, TC-SCHED-005	Pass
FR-11	Cảnh báo vượt ngân sách, nhắc nhở định kỳ.	TC-BUDG-006, TC-BUDG-007, TC-NOTI-006, TC-SCHED-001, TC-SCHED-002	Pass
FR-12	Biểu đồ xu hướng, cơ cấu chi tiêu.	TC-REPO-001, TC-REPO-002	Pass
FR-13	Chia sẻ ví gia đình (View/Edit).	TC-WALL-006 đến TC-WALL-008 TC-FAMI-001 đến TC-FAMI-011	Pass
FR-14	Phân quyền (Admin, User).	TC-USER-005, TC-USER-006	Pass
FR-15	Admin Panel (Quản lý User, Cấu hình).	TC-USER-005, TC-USER-006 (Chưa có Test Case cho Cấu hình hệ thống Settings)	Pass (Cấu hình vận hành: Chưa kiểm)

Mã yêu cầu	Nội dung	Test case phủ	Kết quả lượt gần nhất
FR-16	Xuất báo cáo Excel, PDF.	TC-REPO-003, TC-REPO-004	Pass

Bảng 24: Ma trận truy vết testcase

5.5. Kết quả và defeat

5.5.1 Kết quả

Mã / Tên Module	Tổng số TC	Pass (Lượt 1)	Fail (Lượt 1)	Pass (Lượt 2)	Fail (Lượt 2)
1. Xác thực & Người dùng (AUTH)	11	11	0	11	0
2. Quản lý Danh mục (CATE)	7	7	0	7	0
3. Quản lý Ví (WALL)	8	8	0	8	0
4. Quản lý Giao dịch (TRAN)	9	9	0	9	0
5. Quản lý Ngân sách (BUDG)	8	7	1	8	0
6. Quản lý Gia đình (FAMI)	11	11	0	11	0
7. Mục tiêu tiết kiệm (SAVE)	7	7	0	7	0
8. Nhập dữ liệu (IMP)	5	4	1	5	0
9. Quản lý Thông báo (NOTI)	7	6	1	7	0
10. Lên lịch tự động (SCHED)	5	4	1	5	0
11. Báo cáo Thống kê (REPO)	4	3	1	4	0
12. Sao lưu Phục hồi (BACK)	3	3	0	3	0
Tổng cộng	85	80	5	85	0

Bảng 25: Bảng kết quả

5.5.2. Defeat

Mã Defect	GitHub Issue	Nhãn (Labels)	Mô tả lỗi phát hiện (Fail ở Lượt 1)	Cách khắc phục (Fix)	Trạng thái
DF-01	#24	bug logic	(TC-BUDG-006) Cảnh báo vượt ngân sách 100% không chạy khi tiêu đúng bằng số tiền giới hạn. Do dùng dấu > thay vì >=.	Chỉnh sửa biểu thức điều kiện thành: totalSpent >= limitAmount.	Đã đóng
DF-02	#27	bug math	(TC-REPO-002) Bị lỗi ArithmeticException: / by zero (Chia cho 0) ở biểu đồ khi tháng trước chưa có giao dịch nào.	Thêm lệnh if kiểm tra: Nếu tiền tháng trước = 0 thì tự gán % tăng trưởng là 100%.	Đã đóng
DF-03	#29	bug api	(TC-IMP-003) Khi mạng chậm, API Gemini phản hồi lâu làm sập luôn luồng import file Excel (HTTP 500).	Thêm khối try/catch bọc hàm gọi AI. Nếu AI lỗi thì tự gán vào danh mục "Chưa phân loại" để file vẫn lưu được.	Đã đóng
DF-04	#30	bug security	(TC-NOTI-005) Lỗi bảo mật: Người dùng A truyền ID lên API có thể xóa được thông báo của người dùng B.	Sửa hàm truy vấn Database: Phải bắt buộc khớp cả id của thông báo VÀ userId của người đang đăng nhập.	Đã đóng
DF-05	#33	bug unit-test	(TC-SCHED-001) Bị lỗi NullPointerException khi chạy Unit Test cho hàm nhắc nhở hoá đơn định kỳ.	Bổ sung hàm mockUser.setId(1L) và gán vào hóa đơn giả lập ở phần Arrange của file Test.	Đã đóng

Bảng 26: Bảng defeat

5.6. Đánh giá đối chiếu mục tiêu

Mục tiêu của đề tài đặt ra tại Chương 1 được đối chiếu với kết quả thực tế đo lường được sau quá trình kiểm thử như sau:

Mục tiêu	Phương pháp đo lường	Kết quả thực tế	Đánh giá
Quản lý toàn diện tài khoản, thu chi	Kiểm tra luồng CRUD của giao dịch, ví, danh mục. Test IDOR bảo mật.	100% Test Case (TC-TRAN, TC-CATE, TC-WALL) Pass. Không có lỗi truy cập chéo.	Đạt. Hệ thống đảm bảo toàn vẹn số dư dòng tiền.

Mục tiêu	Phương pháp đo lường	Kết quả thực tế	Đánh giá
Tự động hóa nhập liệu CSV, giảm thủ công	Upload file CSV 100 dòng. Đo tỷ lệ khớp Rule và tỷ lệ phân loại AI.	Khử trùng lặp chính xác 100%. Thuật toán Hybrid phân loại thành công mà không làm gián đoạn hệ thống.	Đạt. Tiết kiệm đáng kể công sức nhập tay của người dùng.
Phân tích trực quan và cảnh báo kịp thời	Tạo giao dịch vượt ngân sách. Chờ Cron Job chạy đến hạn hóa đơn.	Email và In-app Notification (TC-BUDG-004, TC-SCHED-001) được gửi đi tức thời đúng mức ưu tiên.	Đạt. Giúp người dùng kiểm soát tài chính chủ động.

Bảng 27: Bảng đánh giá đối chiếu mục tiêu

5.7. Phần chưa kiểm thử được

Mặc dù hệ thống đã đáp ứng tốt các kiểm thử về chức năng và logic nghiệp vụ, do giới hạn về thiết bị và khuôn khổ của đề án, một số khía cạnh chuyên sâu dưới đây chưa được kiểm thử toàn diện:

Thứ nhất, chưa kiểm thử chịu tải và hiệu năng (Load/Performance Testing). Hệ thống mới được xác nhận hoạt động mượt mà dưới góc độ người dùng đơn lẻ. Nhóm chưa áp dụng các công cụ như JMeter hay k6 để giả lập tình huống hàng trăm người dùng cùng lúc đẩy file CSV lên server, nhằm đo đặc thời gian phản hồi (Response Time) và đánh giá giới hạn chịu tải (Rate Limit) của API Google Gemini miễn phí.

Thứ hai, chưa triển khai kiểm thử giao diện tự động (Automated E2E Testing). Toàn bộ các thao tác kiểm thử trên giao diện ReactJS (như bấm nút, xem biểu đồ, kéo thả file) hiện tại đều được thực hiện thông qua con người (Manual). Việc xây dựng các kịch bản kiểm thử tự động bằng Cypress hoặc Playwright để tự động hóa trình duyệt sẽ là hướng đi cần thiết để tăng tính chuyên nghiệp nếu hệ thống được triển khai thương mại hóa.

Chương 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Đề án đã hiện thực hóa thành công một hệ thống quản lý tài chính của cá nhân và gia đình toàn diện, đáp ứng sát với các mục tiêu đã đề ra tại Chương 1. Về mặt kỹ thuật,

hệ thống đã xây dựng hoàn chỉnh theo kiến trúc Client-Server, với phía máy chủ sử dụng Spring Boot và phía máy khách sử dụng ReactJS, kết hợp cùng cơ sở dữ liệu MySQL và được đóng gói triển khai chuẩn hóa qua Docker.

Đối chiếu với các mục tiêu ban đầu, hệ thống đã đạt được những kết quả cụ thể sau:

Thứ nhất, giải quyết triệt để bài toán nhập liệu thủ công bằng cách xây dựng tính năng import sao kê từ tệp CSV và Excel. Hệ thống không chỉ đọc và chuẩn hóa dữ liệu linh hoạt mà còn áp dụng thuật toán đối chiếu 4 tham số (ví, ngày, số tiền, nội dung) để loại bỏ 100% các giao dịch bị trùng lặp.

Thứ hai, chuyển khai thành công cơ chế phân loại giao dịch lọc qua bằng bộ quy tắc khóa từ vựng, sau đó mới gom các giao dịch chưa nhận diện được thành một lô để gửi đến Google Gemini API. Phương pháp này giúp hệ thống đạt độ chính xác cao trong phân loại nhưng vẫn tối ưu được chi phí và thời gian chờ.

Thứ ba, xây dựng được một công cụ kiểm soát tài chính chủ động thông qua tính năng ngân sách và mục tiêu tiết kiệm. Hệ thống lập lịch ngầm hoạt động ổn định, tự động theo dõi và phát đi cảnh báo qua email cũng như thông báo trong ứng dụng ngay khi người dùng có giao dịch làm vượt ngưỡng chi tiêu an toàn hoặc khi hóa đơn đến hạn.

Cuối cùng, hệ thống đã vượt qua các kịch bản kiểm thử nghiêm ngặt, đảm bảo tính toàn vẹn của dữ liệu số dư và không tồn tại lỗ hổng truy cập chéo dữ liệu giữa các người dùng trong chức năng chia sẻ ví gia đình.

6.2. Hạn chế còn tồn tại

Bên cạnh những kết quả đạt được, phiên bản hiện tại của hệ thống vẫn còn một số hạn chế ảnh hưởng đến trải nghiệm người dùng:

Hạn chế về tính liền mạch trong khâu nhập liệu:

Mô tả: Người dùng vẫn phải đăng nhập vào ứng dụng ngân hàng, xuất tệp sao kê, lưu về máy rồi tải lên hệ thống.

Nguyên nhân: Do giới hạn về mặt pháp lý và chi phí đối với một đề án sinh viên, hệ thống chưa thể kết nối trực tiếp với API ngân hàng mở (Open Banking) của các tổ chức tài chính.

Ảnh hưởng: Quy trình này làm gián đoạn trải nghiệm người dùng, đặc biệt với những người không thường xuyên sao lưu hoặc cập nhật dữ liệu thu chi.

Hạn chế về xử lý tác vụ trí tuệ nhân tạo:

Mô tả: Khi người dùng tải lên tệp sao kê có nhiều giao dịch chưa được định nghĩa từ khóa, hệ thống có thể mất khá nhiều thời gian để xử lý.

Nguyên nhân: Lỗi gọi đến Gemini API hiện được thực hiện đồng bộ trong quá trình xử lý tệp. Nếu API phản hồi chậm hoặc bị giới hạn lưu lượng, thời gian xử lý sẽ kéo dài.

Ảnh hưởng: Người dùng phải chờ lâu trong quá trình tải tệp, dễ tạo cảm giác hệ thống bị chậm.

Hạn chế về nền tảng tương tác trên thiết bị di động:

Mô tả: Hệ thống hiện là một ứng dụng Web đáp ứng (Responsive Web App), có thể sử dụng trên điện thoại nhưng chưa mang lại trải nghiệm tương tự như ứng dụng di động gốc.

Nguyên nhân: Giao diện được phát triển bằng ReactJS thay vì các nền tảng di động chuyên biệt.

Ảnh hưởng: Hệ thống chưa thể tận dụng đầy đủ các tính năng của hệ điều hành di động, chẳng hạn như đọc thông báo SMS để tự động cập nhật biến động số dư hoặc gửi Push Notification khi ứng dụng đang đóng.

6.3. Hướng phát triển

Dựa vào các hạn chế đã phân tích, hệ thống đề ra ba hướng phát triển cụ thể cho các phiên bản tiếp theo:

Tích hợp cổng kết nối ngân hàng mở (Open Banking): Để khắc phục điểm yếu về thao tác tải tệp thủ công, hệ thống sẽ nghiên cứu tích hợp với các dịch vụ tổng hợp dữ liệu tài chính hoặc API của các ngân hàng cho phép. Khi đó, biến động số dư sẽ được đồng bộ hóa về hệ thống theo thời gian thực, người dùng hoàn toàn không cần can thiệp vào quá trình nhập liệu.

Chuyển đổi sang kiến trúc xử lý bất đồng bộ (Asynchronous Processing): Đối với nút thất hiệu năng khi gọi AI, hệ thống sẽ đưa vào sử dụng hàng đợi thông điệp (Message Queue như RabbitMQ hoặc Redis). Khi tệp sao kê được tải lên, hệ thống sẽ ghi nhận và phản hồi thành công ngay lập tức cho giao diện. Tác vụ gọi AI phân loại sẽ được đẩy xuống chạy ngầm (Background Job). Sau khi AI xử lý xong, hệ thống sẽ bắn một thông báo qua WebSocket để cập nhật giao diện cho người dùng.

Phát triển ứng dụng di động gốc: Hệ thống sẽ tận dụng lại toàn bộ nền tảng API hiện có ở phía backend để phát triển thêm một ứng dụng di động bằng React Native hoặc Flutter. Ứng dụng di động sẽ xin cấp quyền đọc thông báo từ thiết bị, tự động nhận diện tin nhắn trừ tiền từ ngân hàng, trích xuất số tiền và đẩy thẳng lên hệ thống, tạo ra trải nghiệm quản lý tài chính hoàn toàn tự động.

6.4. Bài học rút ra

Sau quá trình nghiên cứu và phát triển đề án, bài học lớn nhất nhóm rút ra là vấn đề đảm bảo tính toàn vẹn dữ liệu trong các ứng dụng tài chính (ACID). Ở những phiên bản đầu tiên, hệ thống lấy số dư hiện tại của ví lên bộ nhớ, sau đó cộng hoặc trừ số tiền giao dịch rồi lưu lại. Cách xử lý này dẫn đến sai lệch số dư khi có nhiều giao dịch được import cùng lúc do xảy ra điều kiện tương tranh (race condition). Để khắc phục, nhóm đã thiết kế lại cơ chế cập nhật số dư, chuyển sang thực hiện phép cộng hoặc trừ trực tiếp tại cơ sở dữ liệu. Cách này giúp hạn chế việc nhiều request cùng đọc và ghi đè lên một giá trị số dư.

Bài học thứ hai là việc sử dụng AI cần phù hợp với từng trường hợp. Ban đầu, nhóm dự định sử dụng AI với hầu hết các thao tác phân loại giao dịch. Tuy nhiên, trong quá trình phát triển, nhóm nhận thấy việc gọi AI cho mọi giao dịch làm tăng thời

gian xử lý và sử dụng nhiều tài nguyên hơn cần thiết. Vì vậy, nhóm xây dựng bộ quy tắc dựa trên từ khóa để xử lý các trường hợp thông thường và chỉ sử dụng AI cho những giao dịch không thể phân loại bằng các quy tắc có sẵn. Cách kết hợp này giúp giảm số lần gọi AI và cải thiện thời gian xử lý của hệ thống.

Qua đó, nhận thấy việc lựa chọn công nghệ cần dựa trên nhu cầu thực tế của hệ thống. Một công nghệ hiện đại thì không phải lúc nào cũng là lựa chọn tối ưu nhất. Quan trọng hơn là nhận ra đúng vấn đề và lựa chọn giải pháp mà đảm bảo hệ thống có thể hoạt động ổn định, hiệu quả và dễ bảo trì.

TÀI LIỆU THAM KHẢO

- [1] R. Bai and W. Khan, "Impact of Financial Literacy, Mental Budgeting and Self Control on Financial Wellbeing: Mediating Impact of Investment Decision Making," PLOS ONE, vol. 18, no. 11, e0294466, Nov. 2023. doi: 10.1371/journal.pone.0294466
- [2] VMware, Inc., *Spring Boot Reference Documentation*. [Online]. Available: <https://docs.spring.io/spring-boot/index.html>
- [3] Meta Platforms, Inc., *React – The library for web and native user interfaces*. [Online]. Available: <https://react.dev/>
- [4] Oracle Corporation, *MySQL 8.4 Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/refman/8.4/en/>
- [5] Postman, Inc., *Postman Docs*. [Online]. Available: <https://learning.postman.com/>
- [6] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *RFC 7519*, Internet Engineering Task Force (IETF), May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519.html>
- [7] Finsify JSC, *Money Lover – Ứng dụng quản lý tài chính cá nhân*. [Online]. Available: <https://moneylover.me/vi/>
- [8] Apple Inc., "Sổ thu chi - Quản lý chi tiêu," *App Store*. [Online]. Available: <https://apps.apple.com/us/app/s%E1%BB%95-thu-chi-qu%E1%BA%A3n-l%C3%BD-chi-ti%C3%AAu/id6444122615>
- [9] Công ty Cổ phần MISA, *Sổ Thu Chi MISA (MISA MoneyKeeper) – Ứng dụng quản lý chi tiêu*. [Online]. Available: <https://sothuchi.misa.vn/>